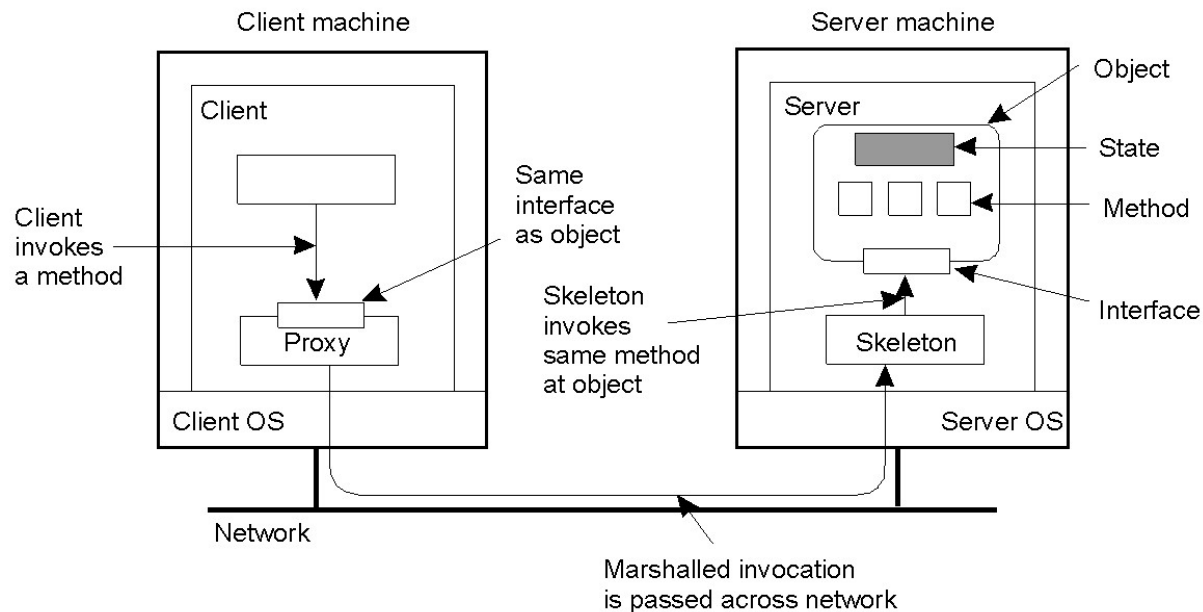


OO pristup -Poziv udaljenih metoda (RMI)

- * RPC na OO način – omogućava pozivanje metoda objekata koji se nalaze na drugom hostu
- * Jedna od najvažnijih k-ka objekta je da on skriva svoju unutrašnjost (podatke- stanja i operacije nad podacima –metode) od spoljnjeg okruženja uz pomoć dobro definisanog interfeisa.
 - ova osobina je ključna za skrivanje heterogenosti u DS
- * objekat može imati skup metoda koje se mogu pozivati sa udaljenih računara.
 - Ovi metodi su definisani u remote inteface fajlu. (objekat može imati i druge metode koje se mogu pozivati samo lokalno)
 - Ne postoji ni jedan drugi način da se pristupi podacima unutar objekta osim preko metoda koji su dostupni preko interfeisa
 - Interfeis predstavlja grupu metoda sa praznim telom (interfeis sadrži samo definicije metoda)
 - samo server ima implementacioni kod za objekat (i za lokalne i za udaljene metode)
 - Pristup objektima preko interfeisa je ključan faktor za DS:
 - Interfeisi mogu biti locirani na jednoj mašini a objekat na drugoj: ovakva organizacija poznata je pod nazivom distribuirani objekti ili udaljeni objekti.
 - » Objekat nije distribuiran (on se nalazi na jednoj mašini), a distribuiran je interfeis.

Distribuirani objekti (nast.)

- * Šta se dešava kada proces na jednoj mašini pozove metod (RMI – Remote Method Invocation) objekta koji se nalazi na drugoj mašini?



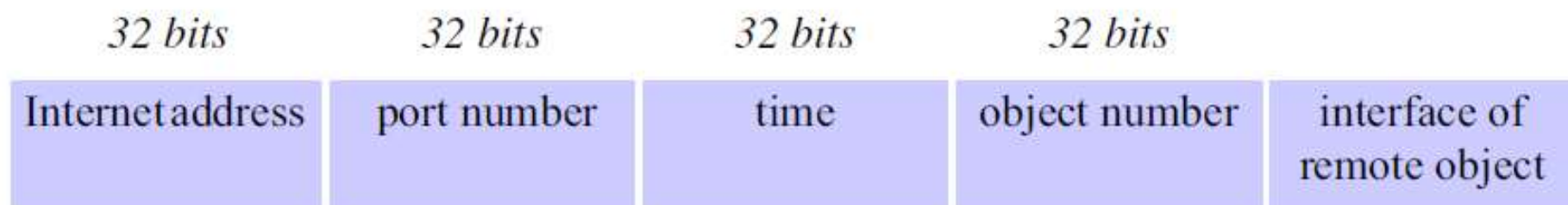
-Poziv se prosleđuje proxy-ju. Proxy je predstavnik server objekta u adresnom prostoru klijenta

- Proxy implementira isti interfeis kao i server (ali na drugi način)
- Proxy je analogan klijent stub-u: poziv metoda pakuje u poruku i prosleđuje serveru (poruka sadrži referencu udaljenog objekta, identifikator metoda i parametre poziva)
- Objekat se nalazi na serveru na kome se nalazi isti interfeis kao na klijent strani
- Dolazni poziv se prvo prosleđuje serverskom stub-u (skeleton), koji raspakuje poruku u poziv odgovarajućeg metoda na server strani

* Kada klijent pozove metod udaljenog objekta, poziv se prosledjuje serverskom procesu u kome se nalazi udaljeni objekat.

- ova poruka mora da specificira objekat čiji se metod poziva
 - koristi se identifikator udaljenog objekta koji je jedinstven u čitavom distribuiranom sistemu – referenca udaljenog objekta (remote object reference)
 - referenca udaljenog objekta se prenosi u poruci da bi se odredio objekat čiji se metod poziva
 - referenca udaljenog objekta mora biti takva da se garantuje njena jedinstvenost u prostoru i vremenu
 - jedan od načina da se to postigne je da referenca sadrži
 - IP adresu računara i broj porta procesa koji je kreirao objekat, vreme kreiranja, lokalni broj objekta (koji se inkrementira svaki put kada se objekat kreira u procesu)
 - referenca udaljenog objekta može sadržati i informacije o interfeisu udaljenog objekta (npr. ime interfeisa)

- jedan od načina da se to postigne je da referenca sadrži
 - IP adresu računara i broj porta procesa koji je kreirao objekat, vreme kreiranja, lokalni broj objekta (koji se inkrementira svaki put kada se objekat kreira u procesu)
 - referenca udaljenog objekta može sadržati i informacije o interfeisu udaljenog objekta (npr. ime interfeisa)



RMI Middleware

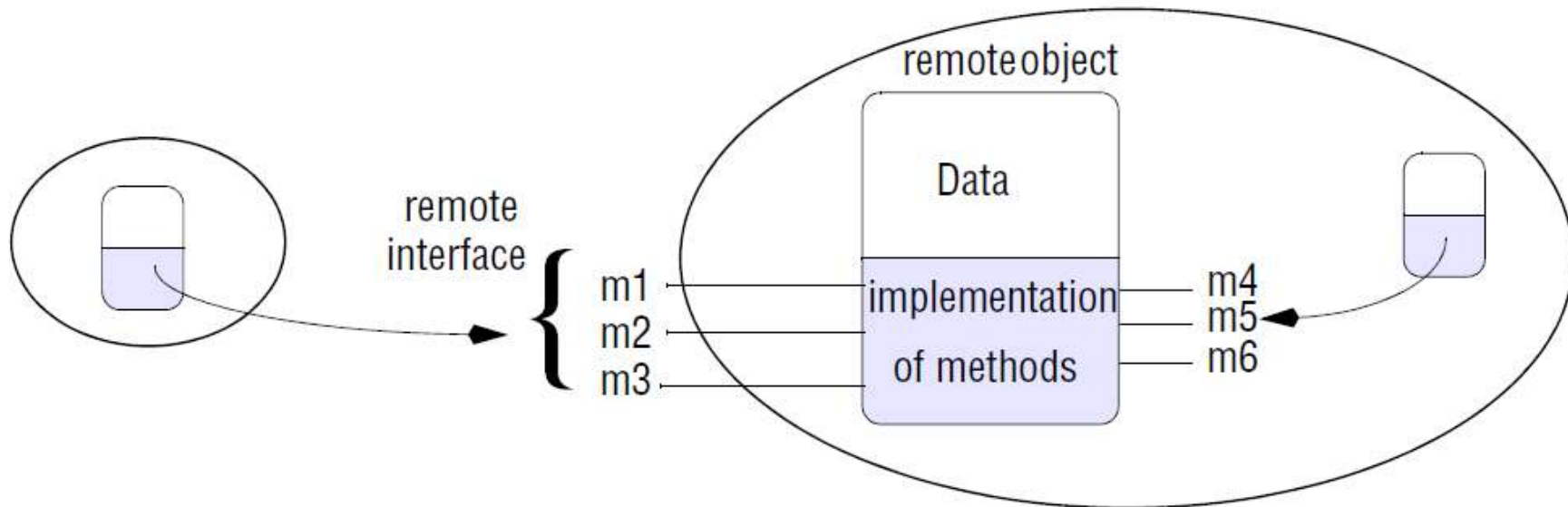
* Primeri middleware baziranih na pozivu udaljenih metoda

- CORBA (Common Object Request Broker Architecture)
- Java RMI
- DCOM (Distributed Component Object Model)

Java RMI

✳ Ideja: logička separacija između interfejsa i objekta omogućava njihovu fizičku separaciju

- Udaljeni interfejs: specificira skup metoda koji se mogu udaljeno pozivati
 - Međutim, unutrašnje stanje udaljenog objekta nije distribuirano!
- Udaljeni objekat: instanca klase koja implementira udaljeni interfejs



Java RMI

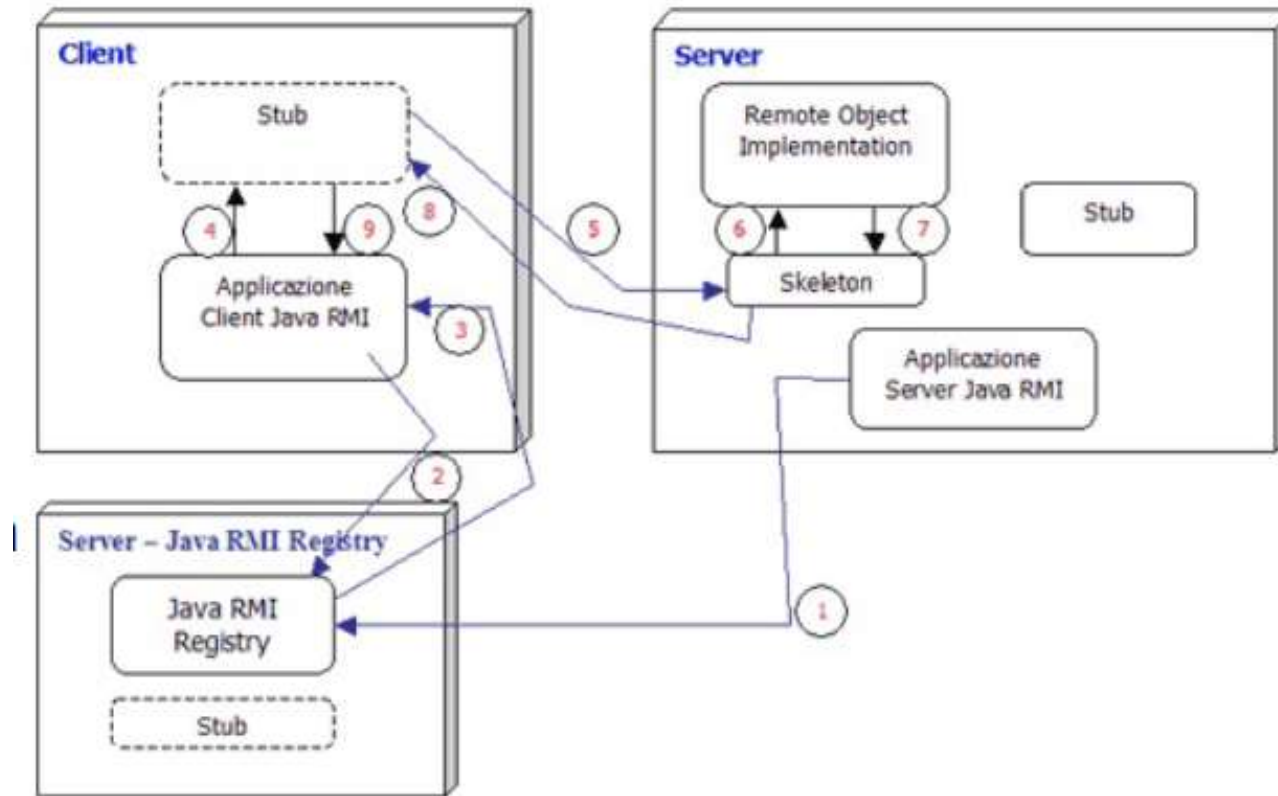
- * Udaljeni interfejsi: Klasa udaljenog objekta implementira metode svog udaljenog interfejsa
- * Objekti u drugim procesima mogu pozvati samo metode udaljenog objekta koje pripadaju njegovom udaljenom interfejsu,
- * Lokalni objekti takođe mogu pozvati metode u udaljenom interfejsu, kao i druge metode implementirane od strane udaljenog objekta.
- * Napomena: udaljeni interfejsi, kao i svi interfejsi, nemaju konstruktore.

povezivanje klijenta i servera

* Kako klijent program doznaje referencu udaljenog objekta?

- koristi se poseban distribuirani servis - *binder*
- u binder-u postoji tabela sa preslikavanjem tekstualnog imena u referencu udaljenog objekta
- server registruje svoje udaljene objekte u binder-u
- klijent koristi binder da pronadje referencu udaljenog objekta

povezivanje klijenta i servera



- RMI registar: binder za Java RMI
- Omogućava serveru da objavi uslugu i klijentu da dobije stub za pristupanje njoj
- Pristupa mu se putem metoda klase Naming, čiji metodi kao argument uzimaju URL-formatiran niz u obliku:

//imeRačunara:port/nazivObjekta

povezivanje klijenta i servera

1. U okviru serverske aplikacije se izvodi registrovanje udaljenog objekta u rmi registru pod određenim imenom
 - vrednost koja se vezana za ime je referenca na udaljeni objekat
2. Kada klijentska aplikacija želi da pristupi udaljenom objektu kontaktira rmi registar sa imenom objekta i dobija referencu udaljenog objekta koja se koristi prilikom instanciranja stuba na klijentskoj strani
3. Klijent poziv metod udaljenog objekta upućuje stub-u
 - Sintaksa za udaljeni poziv je identična lokalnoj

povezivanje klijenta i servera

4. Stub serijalizuje informacije potrebne za poziv (ID metode i ulazne parametre) i šalje ih skeletonu u poruci
5. Skeleton prima poruku i deserijalizuje primljene podatke, prosleđuje poziv ka objektu koji implementira metod, metod se izvršava i generiše rezultat, skeleton serijalizuje povratnu vrednost metoda i šalje je stub-u u poruci
6. Stub prima poruku, deserijalizuje povratnu vrednost i vraća rezultat klijentu

Echo primer: remote interface

- * Udaljeni interfejs nasleđuje `java.rmi.Remote` interfejs.
- * Svaka udaljena metoda:
 - Mora biti deklarirana da baca `RemoteException`
 - Radi rukovanja sa neuspehim komunikacijama
 - Vraća samo 1 rezultat i ima 0, 1 ili više ulaznih parametara
 - Prenos parametara za udaljenu metodu:
 - po vrednosti u slučaju primitivnih tipova podataka (`boolean`, `char`, `int`, ...) ili objekata klase koja implementiraju `java.io.Serializable` interfejs:
 - po referenci u slučaju udaljenih objekata

```
public interface EchoInterface
    extends Remote{
    String getEcho(String echo)
        throws RemoteException;
}
```

Echo primer: server

* Klasa udaljenog objekta

- implementira udaljeni interfejs
- nasleđuje klasu `UnicastRemoteObject`
- `super()` metoda poziva konstruktor klase `UnicastRemoteObject` koji izvršava sve potrebne inicijalizacije da bi omogućio serveru da čeka na zahteve za usluge i da ih uslužuje
- Implementira metodu deklarisanu u interfejsu

```
public class EchoRMIServer
    extends UnicastRemoteObject
    implements EchoInterface{
    //Costruttore
    public EchoRMIServer()
        throws RemoteException
    { super(); }
    // Implement the remote method declared
    // in the interface
    public String getEcho(String echo)
        throws RemoteException
    { return echo; }
```

Echo primer: server

- * U main funkciji serverske aplikacije kreira se instanca serverskog objekta; nakon toga, on može prihvatati udaljene zahteve
 - Server registruje udaljeni objekat pod imenom u RMI registru
 - bind/rebind u klasi Naming (rebind zamenjuje već postojeće povezivanje)
 - Vrš se bind/rebind za svaki serverski objekat koji se registruje, pri čemu se svaki identifikuje svojim logičkim imenom

```
public static void main(String[] args) {  
    // Service registration  
    try  
    {EchoInterface serverRMI =  
        new EchoRMIServer();  
        Naming.rebind("EchoService", serverRMI);  
    }  
    catch (Exception e)  
    {e.printStackTrace(); System.exit(1); }  
}
```

Echo primer: klijent

- * Metod se poziva preko interfejs promenljive, dobijene slanjem zahteva za pretraživanjem RMI registra
- * Pretraživanje reference udaljenog objekta se obavlja pozivom lookup metoda klase Naming.
 - Tačnije, dobija se instanca stub-a udaljenog objekta (koristeći pretraživanje i dodeljivanje promenljivoj interfejsa)
- * Poziv udaljene metode
 - Sinhroni blokirajući poziv

```
public class EchoRMIClient
{
    // Start RMI client
    public static void main(String[] args)
    {
        ...
        try
        {
            // Connect to remote RMI service
            EchoInterface serverRMI = (EchoInterface)
            Naming.lookup("EchoService");
            String message, echo;
            message = stdIn.readLine();
            echo = serverRMI.getEcho(message);
            System.out.println("Echo: "+echo+"\n");
        }
    }
}
```

Prenos parametara

* Prenos parametara prilikom poziva udaljenog metoda:

- Po vrednosti: primitivni tipovi podataka i objekti koji su serijalizovani
 - Serijalizovani objekat: Objekat čije se stanje može razlikovati na različitim lokacijama, prenosi se po vrednosti; objekat se serijalizuje, šalje primaocu i deserijalizuje kako bi se izgradila lokalna kopija. Kada se objekat prenosi prema vrednosti, u procesu primaocu se kreira novi objekat. Metode ovog novog objekta mogu se pozivati lokalno, što može rezultirati različitim stanjem novog objekta u odnosu na stanje originalnog objekta u procesu pošiljaocu.
- Po udaljenoj referenci: udaljeni objekti
 - Objekat koji je vezan za lokaciju u kojoj se izvršava (server) prenosi se putem udaljene reference: njegov stub se prosleđuje drugoj strani

Tipovi komunikacija

* Možemo izvršiti podjelu u odnosu na:

* Postojanost (persistency)

- Tranzijentna naspram perzistentne (istrajne) komunikacije

* Sinhronizacija

- Sinhrona naspram asinhrona komunikacija

* Vremenska zavisnost

- Diskretna naspram strimovane komunikacije

Tipovi komunikacija

* Perzistentne (istrajne)

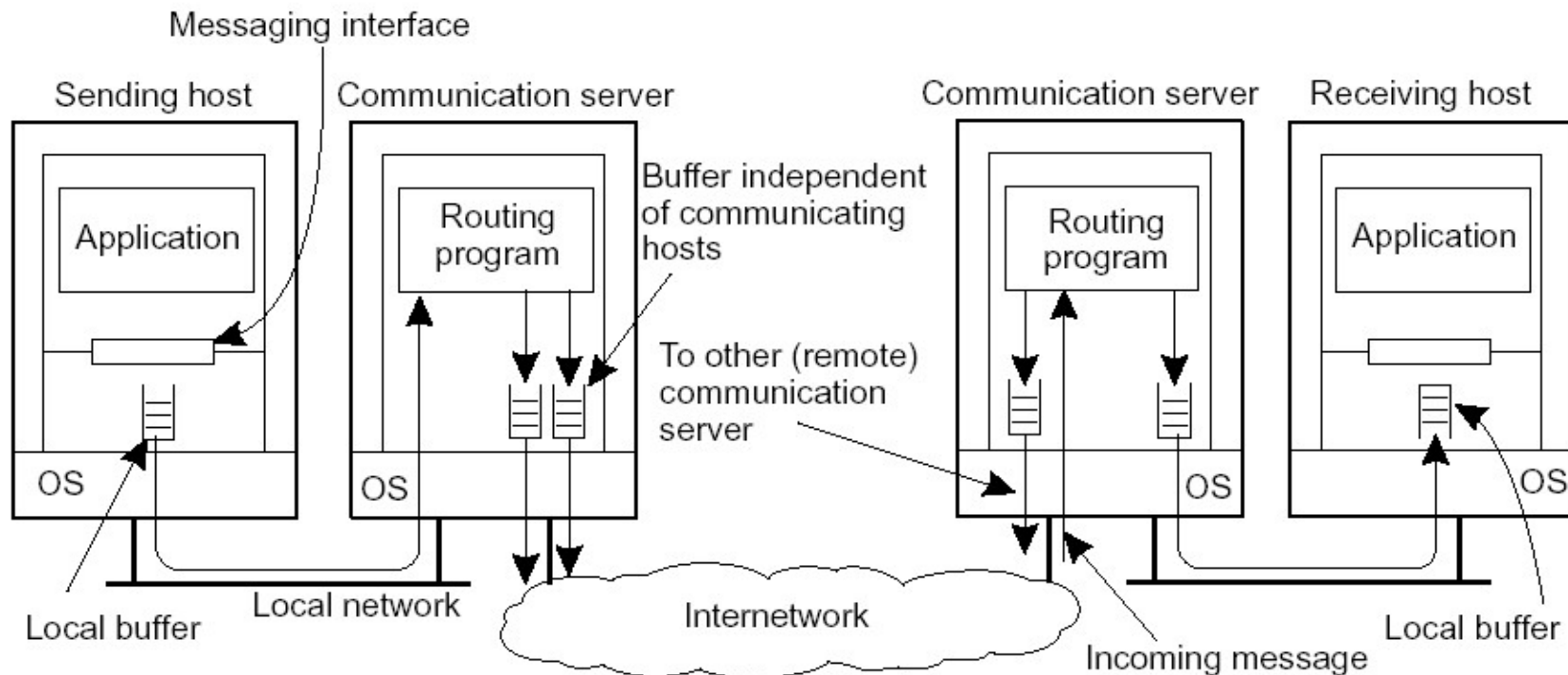
- poruka se pamti u komunikacionom serveru koliko je potrebno da bi se isporučila odredištu

* Tranzijentne

- poruka se odbacuje ako komunikacioni server nije u stanju da je isporuči sledećem serveru ili odredištu

Perzistentne komunikacije

- * poruke koje se prenose se pamte u komunikacionim serverima dok se ne proslede sledećem komunikacionom serveru.
- * izvor ne mora biti aktivan nakon što dostavi poruku komunikacionom serveru
- * prijemnik se ne mora izvršavati u trenutku kada izvor pošalje poruku.
 - Kod perzistentne komunikacije poruka koja treba da se pošalje je zapamćena u komunikacionom serveru onoliko dugo koliko je potrebno da bi bila preneti do prijemnika. (drugog komunikacionog servera)
 - Zbog toga nije neophodno da aplikacija koja je poslala poruku nastavi da se izvršava nakon predaje poruke kom. serveru.
 - Slično, korisnički agent na prijemnoj strani ne mora da se izvršava kada poruka stigne



Primer: E-MAIL

* E-mail sistem je tipičan primer perzistentne (istrajne) komunikacije

- Na hostu se izvršava korisnički agent (aplikacija pomoću koje korisnik može da kreira, šalje i prima poruke)
- Svaki host je povezan na tačno jedan mail server (odgovara komunikacionom serveru)
- Kada korisnički agent prosledi poruku za slanje svom hostu, host je dalje prosleđuje ka svom lokalnom mail serveru u kome se poruka privremeno skladišti
- Mail server obrađuje poruku, pronalazi adresu odredišnog mail servera i uspostavlja vezu sa odredišnim serverom.
- Odredišni server pamti primljenu poruku u ulaznom baferu (mail boxu primaoca)
- Ako je odredišni mail server privremeno nedostupan, lokalni mail server će u svom baferu nastaviti da čuva poruku.
- Korisnički agent pristupa lokalnom mail serveru i kopira poruke iz mail boxa u lokalni bafer.

Tranzientne komunikacije

* Kod tranzientnih komunikacija poruke se skladište samo dok se izvorna i odredišna aplikacija izvršavaju.

- Ako komunikacioni server ne može da isporuči poruku sledećem serveru ili prijemniku, poruka će biti odbačena.

* Ako se agenti ne izvršavaju, poruke se odbacuju.

- Komunikacioni server u ovom slučaju je a store-and-forward ruter.
 - ako ruter ne može da prosledi poruku sledećem ruteru ili odredišnom hostu, poruka se odbacuje (npr. istekao time out)

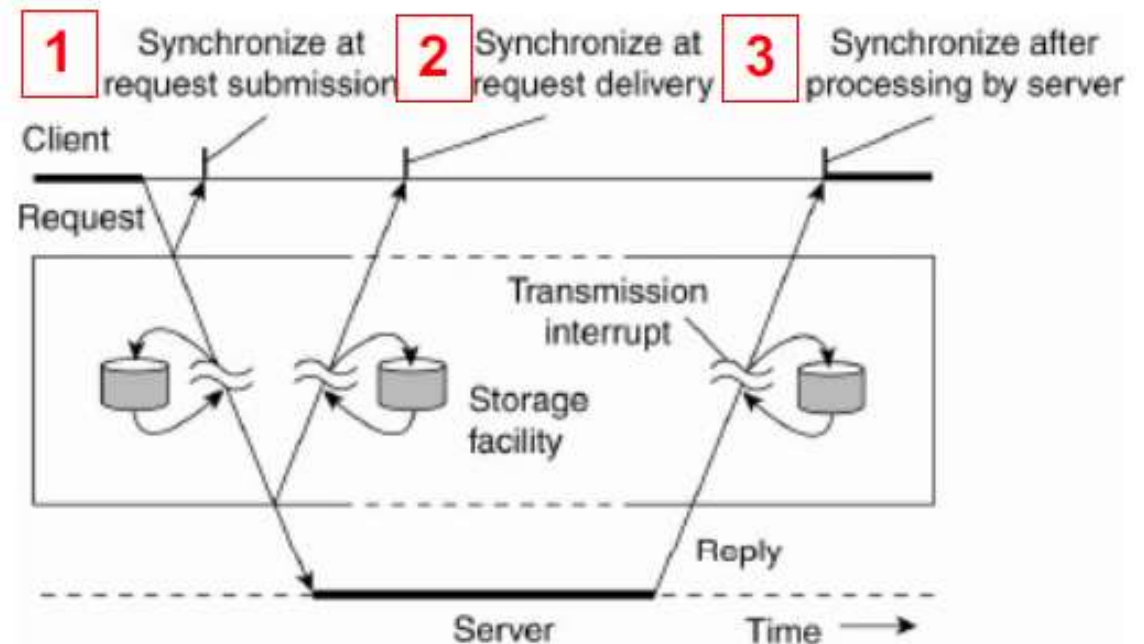
sinhrone i asinhrone komunikacije

- * K-ka asinhronih komunikacija je da pošiljalac nastavlja sa radom odmah nakon što prosledi poruku za slanje
 - Poruka se pamti ili u lokalnom baferu ili u komunikacionom serveru
- * Kod sinhronih komunikacija pošiljalac je blokiran sve dok se ne potvrdi prihvatanje njegovog zahteva.

sinhrone komunikacije

* Jednom kada je poruka poslata, pošiljalac je blokiran sve dok se ne potvrdi prihvatanje njegovog zahteva.

- Koliko dugo je pošiljalac blokiran? Alternative:
 - dok middleware ne preuzme predavanje zahteva
 - dok poruka ne stigne do primaoca (dok zahtev ne bude isporučen primaocu)
 - dok poruka nije potpuno obrađena od strane primaoca (dok zahtev nije potpuno obrađen od strane primaoca, tj. do trenutka kad primalac pošalje odgovor pošiljaocu)



asinhrone komunikacije

- * Nakon što je poruka poslata, pošiljalac nastavlja sa svojim izvršenjem: poruka se privremeno čuva od strane middleware-a dok se ne prenese.

Diskretne i streaming komunikacije

* Diskretna komunikacija:

- Svaka poruka čini kompletan skup informacija.

* Streaming komunikacija:

- Uključuje slanje više poruka, u vremenskom odnosu ili povezanih redosledom slanja, što je potrebno za rekonstrukciju kompletnih informacija.
- Na primer, audio, video.

sinhrone i asinhrone komunikacije

- * Postoji nekoliko mogućih kombinacija ovih tipova komunikacija
 - Perzistentne asinhrone
 - Perzistentne sinhrone
 - Tranzientne asinhrone
 - Tranzientne sinhrone

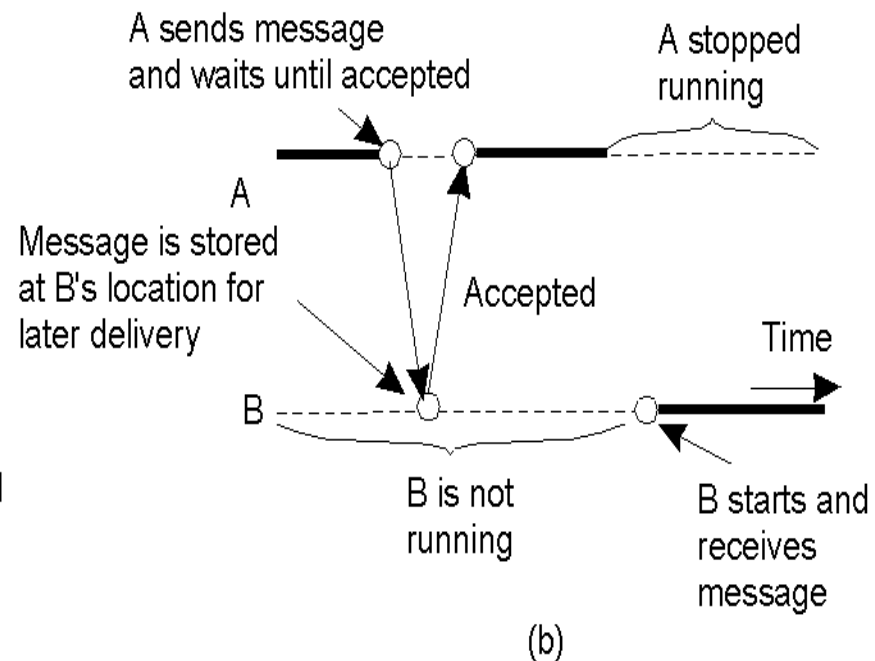
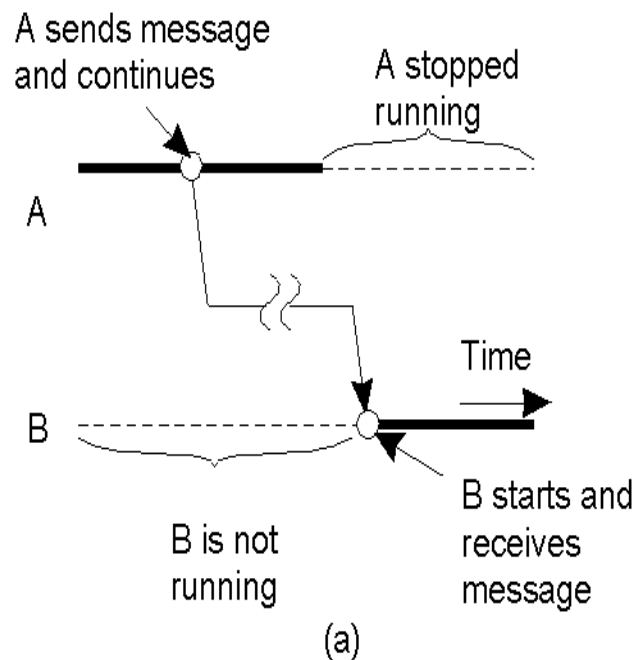
Tipovi komunikacija (nast.)

* a) Perzistentne asinhronne (e-mail, Teams Chat)

- Poruka je zapamćena ili u baferu lokalnog hosta, ili u prvom komunikacionom serveru sve dok se ne isporuči.
 - Pošiljalac nije blokiran dok čeka na isporuku svoje poruke

* b) Perzistentne sinhronne

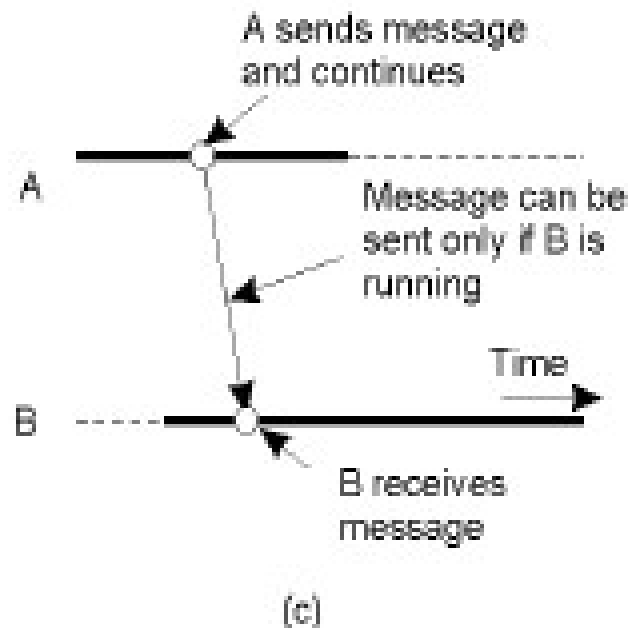
- Poruka mora biti zapamćena u odredišnom hostu da bi se pošiljalac deblokirao.
 - Nije neophodno da se aplikacija na odredišnoj strani izvršava da bi se poruka zapamtila u lokalnom hostu
 - Slabiji zahtev je blokiranje izvora dok se poruka ne zapamti u komunikacionom serveru odredišnog hosta.



Tipovi komunikacija (nast.)

* c) Tranzijentne asinhronone komunikacije

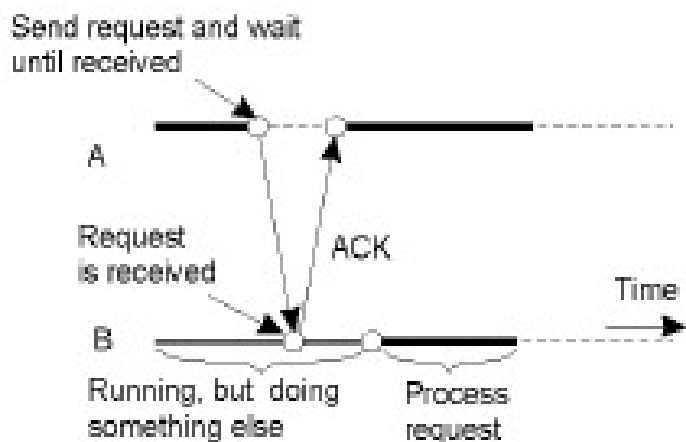
- Poruka se privremeno pamti u lokalnom baferu izvornog hosta, nakon čega aplikacija nastavlja sa izvršenjem
 - Paralelno, komunikacioni sistem prosleđuje poruku do odredišta gde se poruka skladišti u lokalnom baferu
 - Ako prijemnik nije aktivan u trenutku kada poruka stiže do njega, poruka je izgubljena



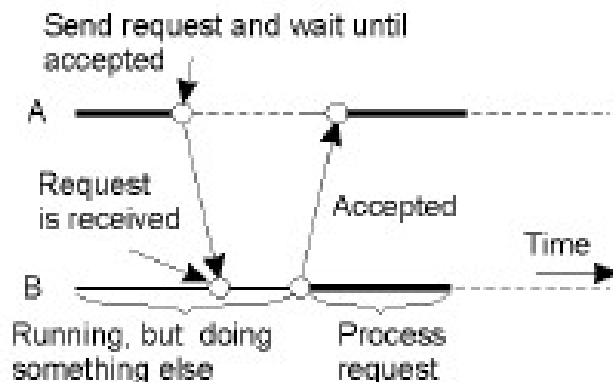
Tipovi komunikacija (nast.)

* Tranzientne sinhronne komunikacije se javljaju u nekoliko oblika

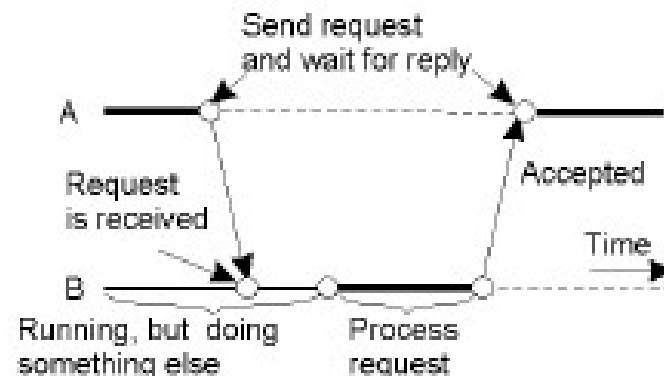
- d) Najslabija forma zahteva blokiranje izvora dok se poruka ne zapamti u baferu odredišnog hosta
 - Kada primi potvrdu izvor se deblokira
- e) izvor je blokiran sve dok se poruka ne prosledi odredišnom procesu za dalju obradu (izvor je blokiran dok se ne isporuči primaocu)
- f) izvor je blokiran sve dok od odredišnog procesa ne primi odgovor (sinhroni RPC i RMI funkcionišu po ovoj šemi)



(d)



(e)



(f)

Middleware bazirani na razmeni poruka (message-oriented communication)

- * Pozivi udaljenih procedura i pozivi udaljenih objekata doprinose sakrivanju komunikacije u distribuiranim sistemima, tj. omogućavaju transparentnost pristupa. Nažalost, ovaj mehanizam nije uvek prikladan.
 - Posebno kada se ne može pretpostaviti da će strana koja prima zahtev izvršavati u trenutku kada je zahtev izdat, potrebne su alternativne komunikacione usluge.
- * Isto tako, inherentna sinhrona priroda RPC-ova, po kojoj je klijent blokiran dok se njegov zahtev ne obradi, može biti zamenjena nečim drugim.

Middleware bazirani na razmeni poruka (message-oriented communication)

* Taj mehanizam se bazira na razmeni poruka.

- Ovde ćemo da se fokusiramo na komunikaciju u distribuiranim sistemima orijentisanu ka razmeni poruka
- Postoje sistemi za razmenu poruka koji pretpostavljaju da strane koje učestvuju u komunikaciji izvršavaju u trenutku komunikacije.
- Postoje sistemi za razmenu poruka koji omogućavaju procesima da razmenjuju poruke, čak i ako druga strana nije aktivna u trenutku pokretanja komunikacije.

Middleware bazirani na razmeni poruka (message-oriented communication)

- * Prvi sistemi su se bazirali na tranzijentnim komunikacijama
 - Pružali su podršku za komunikaciju između procesa koji se jednovremeno izvršavaju.
- * Ovakve komunikacije često nisu adekvatne ako se ima u vidu potencijalno velika geografska distribucija procesa.
 - Potreba za perzistentnim komunikacionim uslugama se pojavila kada je bilo potrebno integrisati aplikacije koje su se izvršavale na geografski jako udaljenim računarima (WAN).

Tranzientne komunikacije: MPI

✱ MPI (Message Passing Interface) middleware sistem koji se bazira na prenosu poruka.

- Namenjen je za podršku komunikacijama u paralelnim sistemima (klasteri, mreža radnih stanica, multiračunarima, multiprocesorima)
 - Socket-i podržavaju komunikaciju između distribuiranih procesa razmenom poruka, ali su projektovani za TCP/IP protokol stek
 - Nisu pogodni za intrrprocesnu komunikaciju kada se koriste drugi protokli
- Pre pojave ovog standarda, svaki proizvođač paralelnih sistema je nudio svoju varijantu sistema za interprocesorsku komunikaciju razmenom poruka.
 - Javio se problem prenosivosti programa

✱ Pojavila se potreba za definisanjem standarda nezavisnog od hardverske platforme.

- MPI podržava tranzientne komunikacije između procesa.

MPI (nast.)

- * MPI podržava komunikaciju između konkurentnih procesa
 - point-to-point
 - grupna komunikacija
- * Implementiran je kao biblioteka funkcija koje se mogu pozivati iz konvencionalnih programskih jezika, kao što su Fortran, C, C++, MPI 3 standard preko 440 funkcija
- * MPI podrazumeva da se komunikacija obavlja u okviru poznate grupe procesa.
 - Svakoj grupi se dodeljuje identifikator
 - Svakom procesu unutar grupe se takođe dodeljuje (lokalni) identifikator.
 - Par (identifikator_grupe, identifikator_procesa) na jedinstveni način identifikuje izvor ili odredište poruke i koristi se umesto transportnih adresa.
 - U sistemu može postojati više grupa procesa (koje se mogu i preklapati) koje obavljaju izračunavanje i koje se izvršavaju u isto vreme.
- * Srž MPI čine funkcije za razmenu poruka između procesa.
 - MPI podržava skoro sve oblike tranzientnih komunikacija.

MPI (nast.)

* `mpiexec -n 10 hello.exe`

Hello World

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    int rank, size;
    MPI_Init(&argc, &argv);

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    printf("Hello world from %d of %d\n", rank, size)

    MPI_Finalize();
    return 0;
}
```

```
Hello from 1 of 10
Hello from 2 of 10
Hello from 6 of 10
Hello from 8 of 10
Hello from 9 of 10
Hello from 0 of 10
Hello from 5 of 10
Hello from 3 of 10
Hello from 7 of 10
Hello from 4 of 10
```

MPI (nast.)

- * MPI middleware održava svoje sopstvene bafere, a sinhronizacija je često povezana sa trenutkom kada su potrebni podaci kopirani iz pozivajuće aplikacije u middleware.
- * Tranzijetna asinhrona komunikacija podržana je operacijom MPI_Bsend. Pošiljalac predaje poruku za prenos, koja se obično prvo kopira u lokalni bafer u sistemu MPI.
 - Kada se poruka iskopira, pošiljalac nastavlja. Lokalni sistem za izvršavanje MPI će ukloniti poruku iz svog lokalnog bafera i brinuti se o prenosu čim primalac pozove operaciju prijema.
- * Postoji i blokirajuća operacija slanja, nazvana MPI_Send, čija semantika zavisi od implementacije.
 - Operacija MPI_Send može blokirati pozivaoca dok određena poruka nije kopirana u sistem za izvršavanje MPI na strani primaoca(MPI bafer odredišnog hosta), ili dok primalac nije pokrenuo operaciju prijema.
- * Sinhronizovana komunikacija, pri kojoj se pošiljalac blokira dok se njegov zahtev ne prihvati za dalju obradu(dok ne krene prijem), dostupna je putem operacije MPI_Ssend.
- * Podržana je i najjača forma sinhronizovane komunikacije: kada pošiljalac pozove MPI_Sendrecv, šalje zahtev primaocu i blokira se dok ovaj ne vrati odgovor. Ova operacija suštinski odgovara sinhronom RPC-u.

MPI (nast.)

- * Sa `MPI_Isend`, pošiljalac predaje pokazivač na poruku, nakon čega se MPI sistem brine o komunikaciji. (nema potrebe za kopiranjem poruke iz korisničkog bafera u bafer lokalnog MPI sistema)
 - Pošiljalac odmah nastavlja. Kako bi se sprečilo prepisivanje poruke pre završetka komunikacije, MPI nudi operacije za proveru završetka, ili čak blokiranje ako je potrebno.
- * Slično tome, sa `MPI_Issend`, pošiljalac takođe predaje samo pokazivač MPI sistemu za izvršavanje.
- * Operacija `MPI_Recv` se poziva za prijem poruke; blokira pozivaoca dok poruka ne stigne.
- * Operacija `MPI_Irecv`, pri kojoj primalac naznačava da je spreman da prihvati poruku.
 - Primalac može proveriti da li je poruka zaista stigla, ili blokirati dok ne stigne.

MPI komunikacione primitive

Primitive	Značenje
MPI_bsend	Kopiraj izlaznu poruku u lokalni bafer MPI sistema i nastavi sa radom(neblokirajuća primitiva- tr. asinhr. kom.) (varijanta c) u prethodnom primeru)
MPI_send	Pošalji poruku i čekaj dok se ne iskopira u bafer odredišnog hosta (izvor se blokira dok poruka ne bude smeštena u MPI bafer odredišnog hosta – varijanta d).
MPI_ssend	Pošali poruku i čekaj dok ne kreće prijem (izvor je blokiran dok ne kreće prijem, varijanta e))
MPI_sendrecv	Pošali poruku i čekaj odgovor (izvor je blokiran dok ne stigne odgovor iz odredišta –varijanta f)) – ponaša se isto kao RPC
MPI_issend	Prosledi referencu na izlaznu poruku i nastavi sa izvršenjem (nema potrebe za kopiranjem poruke iz korisničkog bafera u bafer lokalnog MPI sistema) Da bi se izbeglo prepisivanje poruke postoji mogućnost provere da li je komunikacija završena ili se blokiranja dok se ne završi
MPI_issend	Prosledi referencu na izlaznu poruku i čekaj dok ne kreće prijem
MPI_recv	Prihvati poruku; prijemnik se blokira dok ne stigne poruka
MPI_irecv	Prijemnik je spreman da prihvati poruku – prijemnik može da proveri da li je poruka stigla ili da čeka dok poruka ne stigne.

Zašto ovoliko različitih primitiva?

-Pružaju mogućnost projektantu programa da optimizira performanse

Perzistentne komunikacije: sistemi sa redovima poruka (Message queuing systems-MQS)

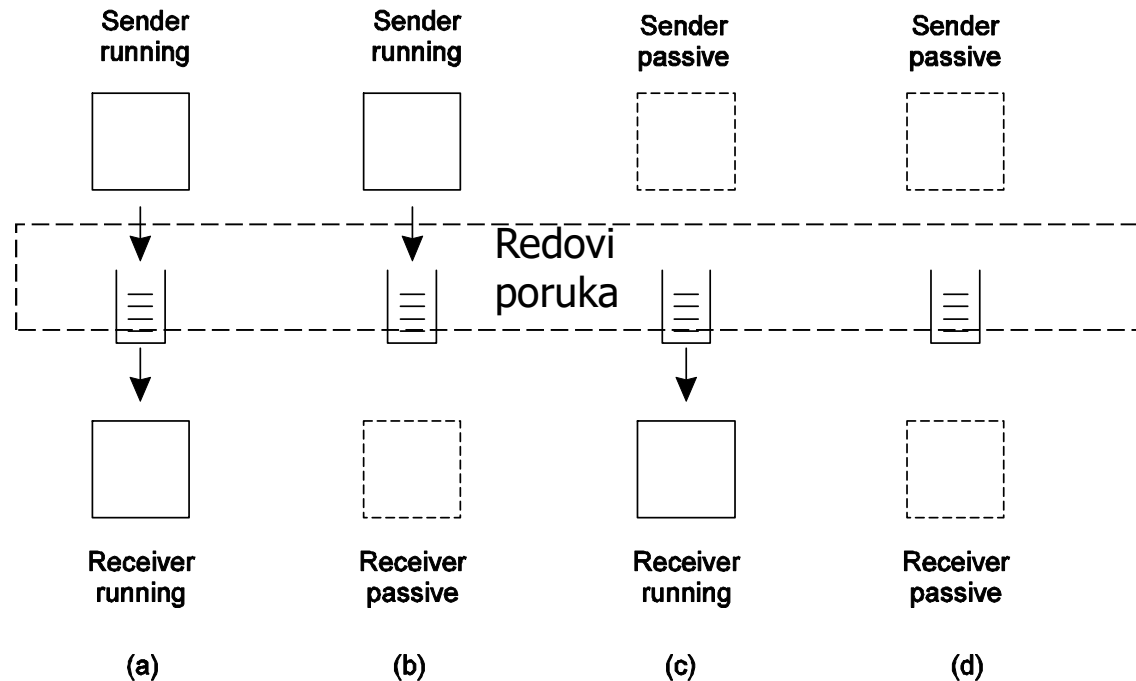
* MQS su važna klasa middleware sistema zasnovana na slanju poruka

- pružaju podršku perzistentnim asinhronim komunikacijama
- namenjeni su aplikacijama kod kojih je dozvoljeno da prenos poruka traje i nekoliko minuta (umesto nekoliko sec ili msec)
- Ovi sistemi poseduju smeštajne kapacitete za usputno pamćenje poruka i ne zahtevaju ni od izvora ni od odredišta da budu aktivni za vreme prenosa poruka.

* Aplikacije komuniciraju smeštanjem poruka u posebne redove čekanja (queues).

- Pošiljaocu se samo garantuje da će njegova poruka biti upisana u red čekanja primaoca.
- ne pružaju se garancije da li će (i kada će) poruka biti pročitana od strane primaoca.
- Ne postoji potreba da se pošiljalac izvršava u trenutku kad se poruka preuzima od strane primaoca
- Pošiljalac i primalac se izvršavaju potpuno nezavisno jedan od drugog
- Jednom kad se poruka isporuči u red čekanja, ostaje tamo, dok se ne ukloni, nezavisno od toga da li se pošiljalac ili primalac izvršavaju
- Ovo nam omogućava četiri kombinacije u odnosu na način izvršenja pošiljaoca i primaoca koji su prikazani na slici

Sistemi sa redovima poruka (Message queuing systems)



- a) Pošiljalac i primalac se izvršavaju za celokupno vreme prenosa poruke
- b) Samo se pošiljalac izvršava, dok je primalac pasivan, tj. u stanju kad isporučivanje poruke nije moguće. I pored toga, pošiljalac i dalje može da šalje poruke
- c) Ovde je prikazan pasivni pošiljalac i aktivni primalac (primalac koji se izvršava)
U ovom slučaju, primalac može čitati poruke koje su mu poslate, ali nije neophodno da pošiljalac poruke bude aktivan
- d) Opisuje situaciju gde sistem skladišti (i moguće prenosi) poruke dok su i pošiljalac i primalac pasivni. Neki smatraju da kad je ova konfiguracija podržana, sistem zaista podržava perizistentnu komunikaciju.

Sistemi sa redovima poruka (Message queuing systems)

Veoma bitan aspekt iz perspektive middleware sistema je da su poruke u sistemu pravilno adresirane.

Adresiranje je omogućeno obezbeđivanjem jedinstvenog imena odredišnog reda u sistemu

Osnovni interfejs koji je na raspolaganju aplikacijama za pristup redu poruka je veoma jednostavan i sadrži 4 operacije: put, get, poll i notify.

Operation	Description
put	Append a message to a specified queue
get	Block until the specified queue is nonempty, and remove the first message
poll	Check a specified queue for messages, and remove the first. Never block
notify	Install a handler to be called when a message is put into the specified queue

Sistemi sa redovima poruka (Message queuing systems)

Put operacija se poziva od strane pošiljaoca koji prosleđuje poruku sistemu koja treba biti dodata u specificirani red. Ovo je neblokirajući poziv.

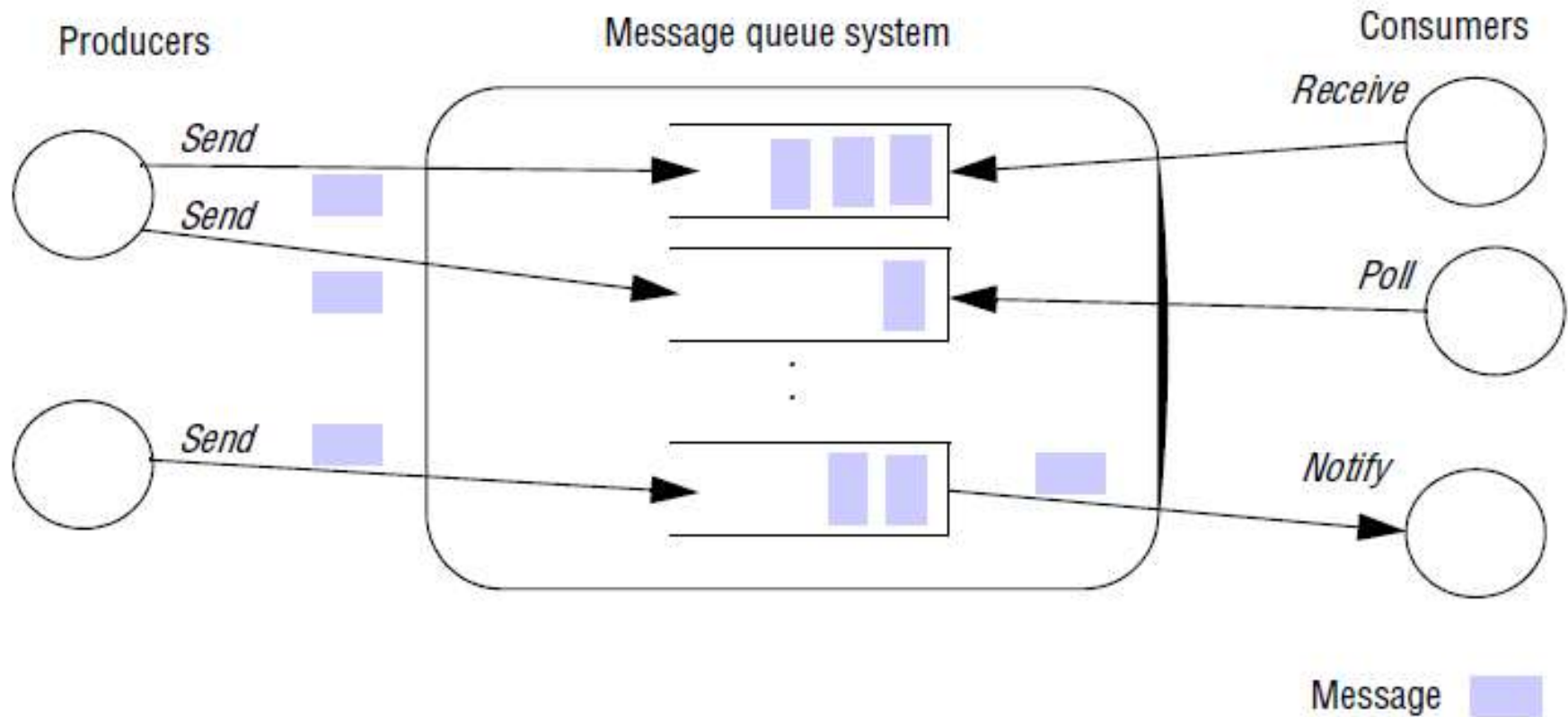
Get operacija predstavlja blokirajući poziv koja omogućava autorizovanom procesu da pribavi iz specificiranog reda poruku koja je najduže u njemu. Proces se blokira ako je red prazan. Kad red nije prazan, proces čita poruku iz reda. Varijacije ovog poziva omogućavaju traženje specifične poruke u redu (pretraživanje poruke po sadržaju)

Poll operacija je neblokirajuća varijanta get operacije. Ako je red prazan, ili specifična poruka ne postoji pozivajući proces nastavlja sa izvršenjem. Proces primalac proverava uslove i ako je neki zadovoljen, čita poruku. U suprotnom nastavlja dalje (ne blokira se)

Notify operacija. Omogućeno je procesu da instalira handler koji se automatski poziva kad god je poruka dodata u red. Može se iskoristiti i da automatski startuje proces koji će pribavljati poruke iz reda, ako se nijedan proces u tom trenutku ne izvršava

Ovaj pristup je često implementiran kroz demon na strani primaoca koji neprekidno nadgleda (osluškuje) dolazeće poruke u red i upravlja njima

Sistemi sa redovima poruka (Message queuing systems)

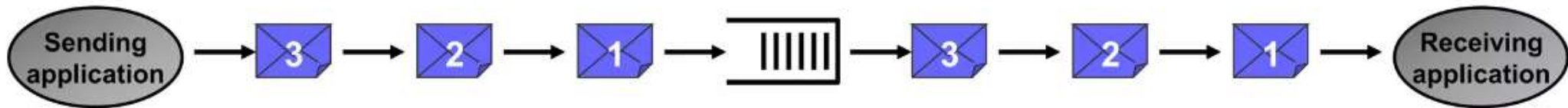


Više procesa može slati poruke u isti red, i isto tako, više primaoca mogu uklanjati poruke iz reda. Politika upravljanja redom je uglavnom FIFO, ali većina implementacija redova poruka takođe podržava koncept prioriteta, pri čemu se poruke višeg prioriteta dostavljaju prvo.

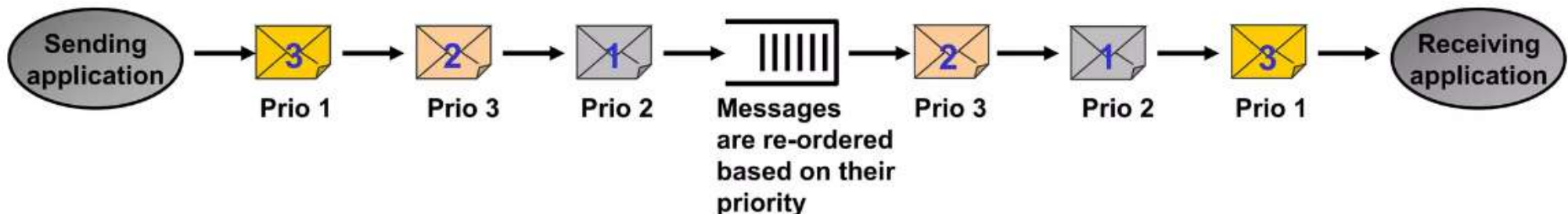
Sistemi sa redovima poruka (Message queuing systems)

Politika upravljanja redom- FIFO (first-in, first-out):

Sve poruke imaju isti prioritet, poruke se isporučuju u redosledu kako poslate



Politika upravljanja redom po prioritetu: Poruke se baferuju u red u redosledu kako su poslate, a onda preuređuju po prioritetu



Sistemi sa redovima poruka

❖ Scenario:

- ❖ Potrebna je platforma za elektronsku trgovinu gde kupci mogu da postavljaju porudžbine za različite proizvode, a ove porudžbine treba efikasno obraditi od strane različitih backend servisa.

❖ Implementacija:

❖ Proizvođač (Kupac):

- ❖ Kupci postavljaju porudžbine putem veb ili mobilne aplikacije. Kada kupac pošalje porudžbinu, ona se dodaje u red poruka. Svaka porudžbina je predstavljena kao poruka u redu.

Sistemi sa redovima poruka

❖ Red poruka:

- ❖ Red poruka deluje kao posrednik između kupca koji postavlja porudžbinu i backend servisa odgovornih za obradu porudžbina. Red prima porudžbine od kupaca i zadržava ih dok ih ne preuzmu i obrade backend servisi. Red osigurava da porudžbine budu obrađene na pouzdan način, čak i tokom perioda visokog saobraćaja.

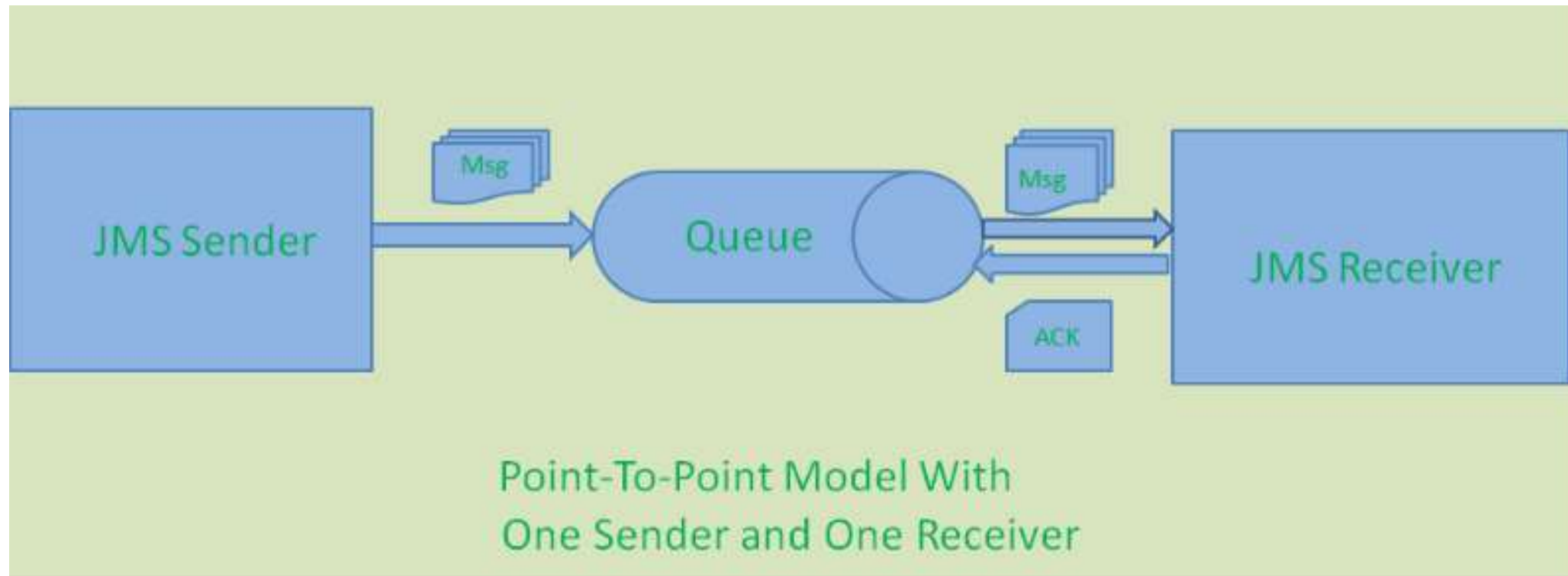
❖ Potrošači (Backend servisi):

- ❖ Backend servisi su odgovorni za obradu porudžbina deluju kao potrošači reda poruka. Ovi servisi kontinuirano prate red poruka za nove porudžbine. Kada poruka porudžbine bude preuzeta iz reda, odgovarajući servis obrađuje porudžbinu.

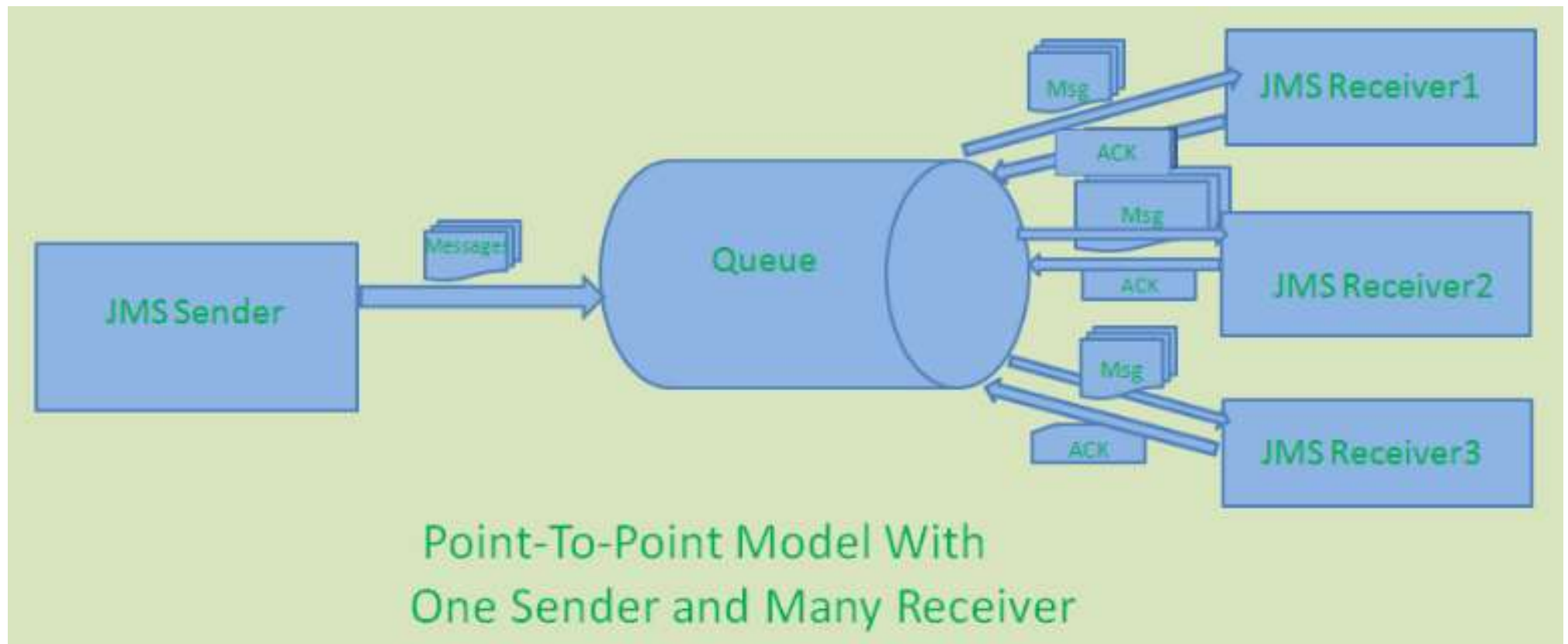
Sistemi sa redovima poruka

- ❖ Sistemi sa redovima poruka podržavaju Point-to-Point model razmene poruka koji može biti implementiran na sledeće načine:
 - ❖ Jedan pošiljalac i jedan primalac povezani preko reda
 - ❖ Jedan pošiljalac i više primalaca povezani preko reda
 - ❖ Više pošiljalaca i jedan primalac povezani preko reda
 - ❖ Više pošiljalaca i više primalaca povezani preko reda
- ❖ Nevezano za to koliki je broj pošiljalaca i primalaca poruka poslata od strane jednog pošiljaoca može biti primljena od strane samo jednog primaoca.
- ❖ U P-to-P modelu poruka od jednog pošiljaoca ne može biti prenetu do više primalaca.

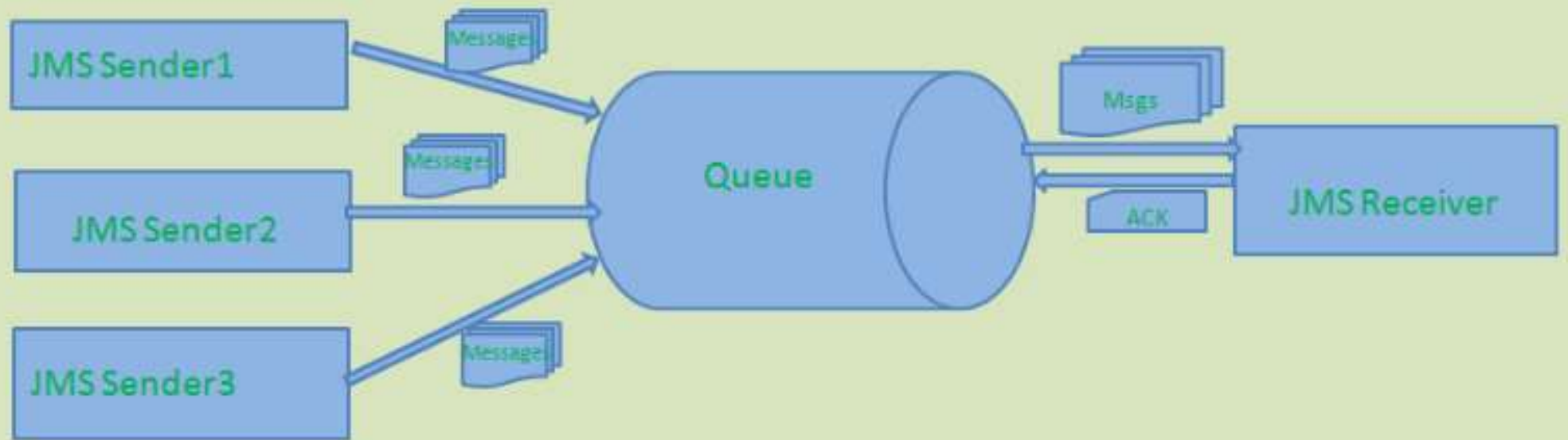
Sistemi sa redovima poruka



Sistemi sa redovima poruka

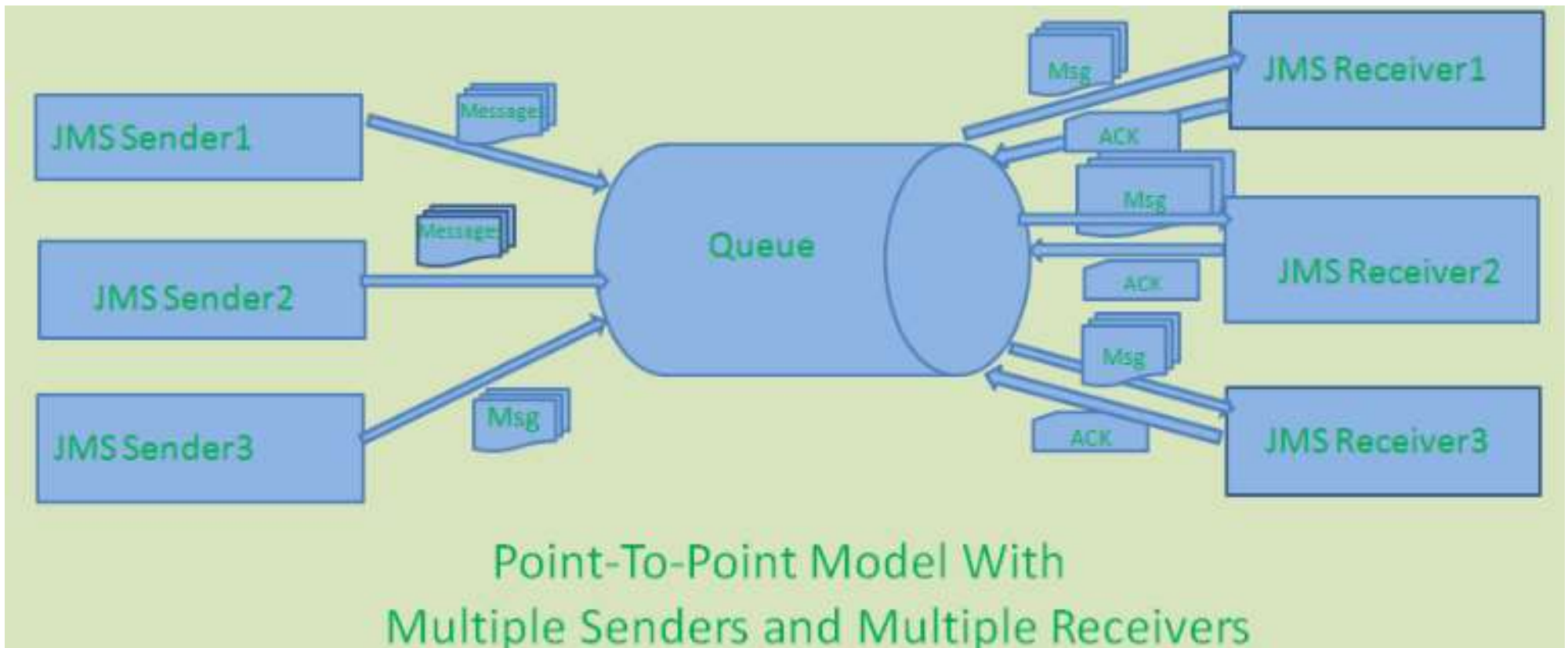


Sistemi sa redovima poruka

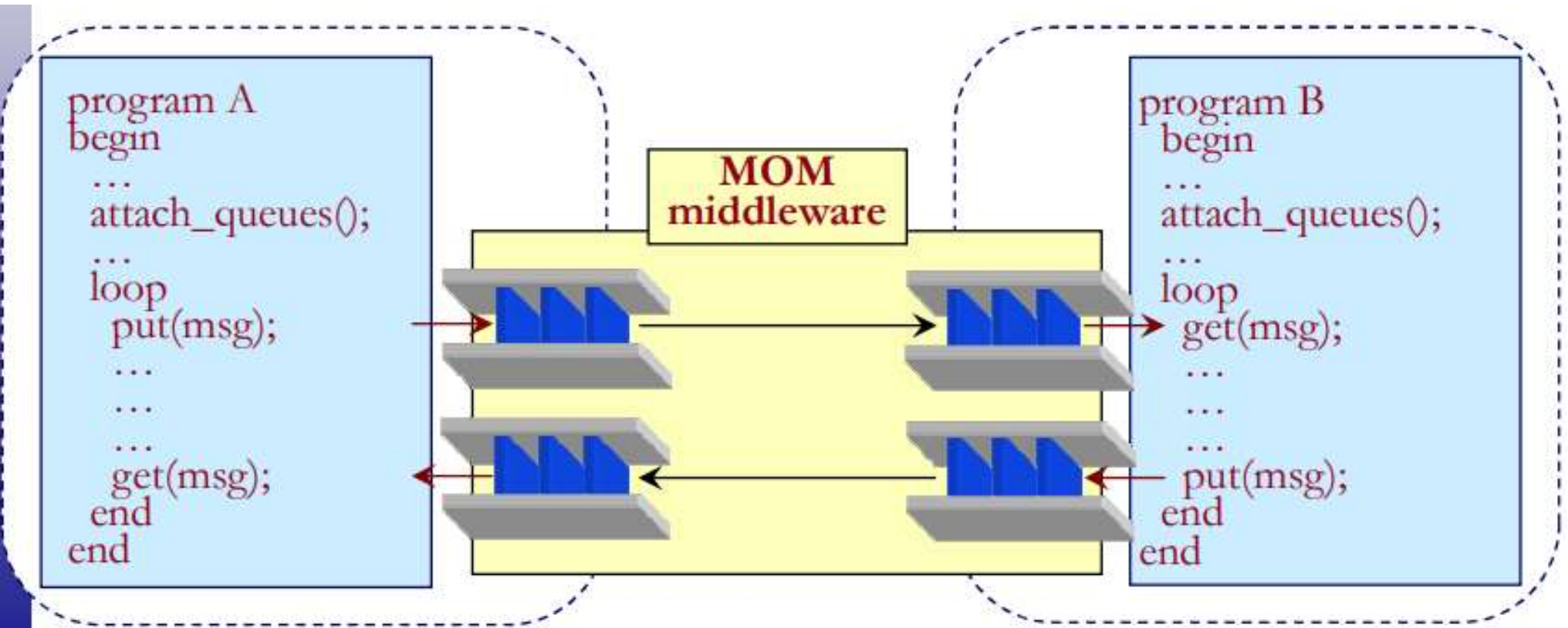


Point-To-Point Model With
Multiple Senders and One Receiver

Sistemi sa redovima poruka



Sistemi sa redovima poruka (Message queuing systems)



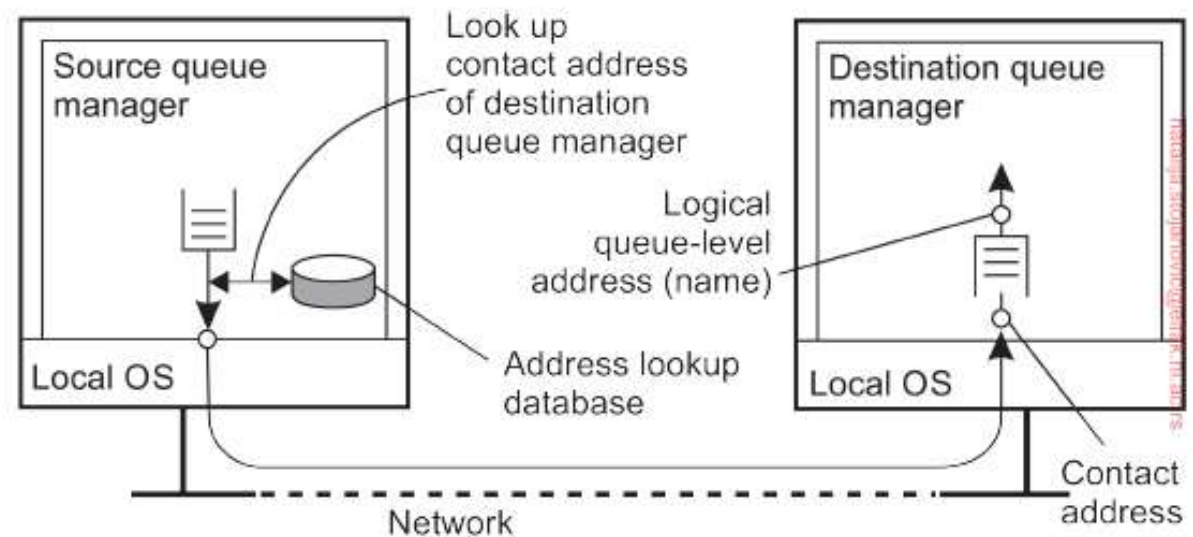
Arhitektura sistema sa redovima poruka (Message queuing systems)

* Kako izgleda opšti sistem za upravljanje redovima poruka?

- Redovima se upravlja pomoću upravljača redova. Upravljač redova je uglavnom zaseban proces.
- Aplikacija može stavljati poruke samo u lokalni red. Isto tako, čitanje poruke je moguće samo izvlačenjem iz lokalnog reda.
- Kao posledica, ako upravljač reda QMA koji upravlja redovima za aplikaciju A radi kao zaseban proces, oba procesa QMA i A će uglavnom biti smešteni na istom računaru.
- Ako aplikacije mogu stavljati poruke samo u lokalne redove, onda je očigledno da će svaka poruka morati da nosi informacije o svom odredištu. Zadatak upravljača redova je da se pobrine da poruka stigne do svog odredišta.

Arhitektura sistema sa redovima poruka (Message queuing systems)

- * 1. pitanje: moramo razmotriti kako je adresiran odredišni red. Očigledno je da bi bilo poželjno da redovi imaju logička, lokacijski nezavisna imena, kako bi se postigla transparentnost lokacije.
- Pretpostavljajući da je upravljač redova implementiran kao zaseban proces, korišćenje logičkih imena podrazumeva da svako ime treba da bude povezano sa kontakt adresom, kao što je (računar, port)-par, i da je preslikavanje imena u adresu lako dostupno upravljaču redova, kao što je prikazano na slici.

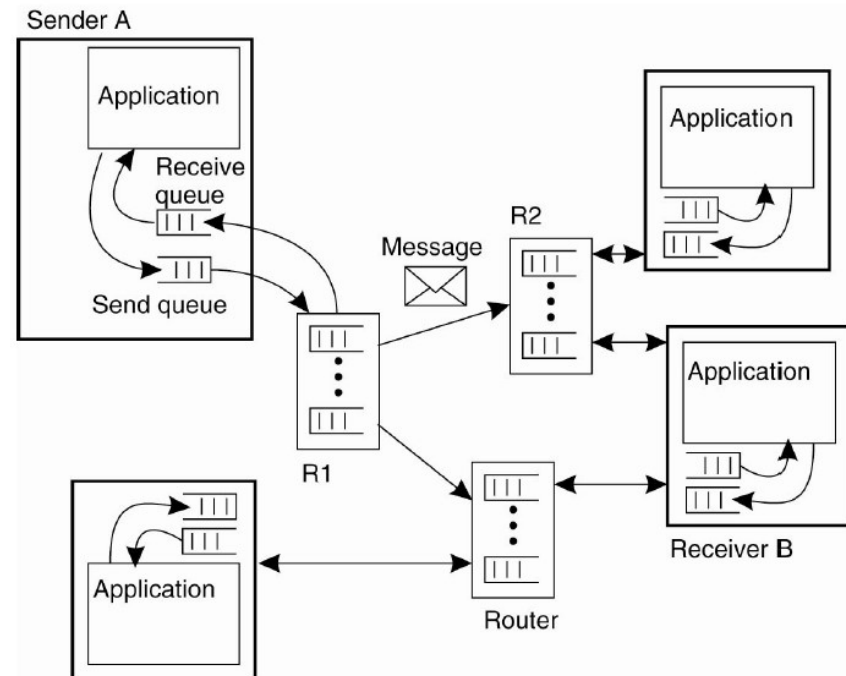


Arhitektura sistema sa redovima poruka (Message queuing systems)

- * 2. pitanje koje treba razmotriti je kako se preslikavanje imena u adresu zapravo čini dostupnim upravljaču redova.
 - Čest pristup je jednostavno implementirati preslikavanje kao tabelu za pretraživanje i kopirati tu tabelu svim upravljačima. Očigledno, to dovodi do problema održavanja, jer svaki put kada se dodaje novi red ili daje ime, mnoge, ako ne i sve tabele, treba ažurirati
- * To nas dovodi do 3. pitanja, koje se odnosi na probleme efikasnog održavanja preslikavanja imena u adresu.
 - To znači da bi kontaktna adresa svakog upravljača redova trebalo da bude poznata svim ostalim. Očigledno je da ćemo, kada se bavimo velikim sistemima za redove poruka, imati problem sa skalabilnošću.
 - U praksi, često postoje posebni upravljači redova koji funkcionišu kao ruteri: oni prosleđuju dolazne poruke drugim upravljačima redova.
 - Na taj način, sistem za redove poruka može postepeno da raste u potpunosti, na nivou aplikacije, preklapajuću mrežu.

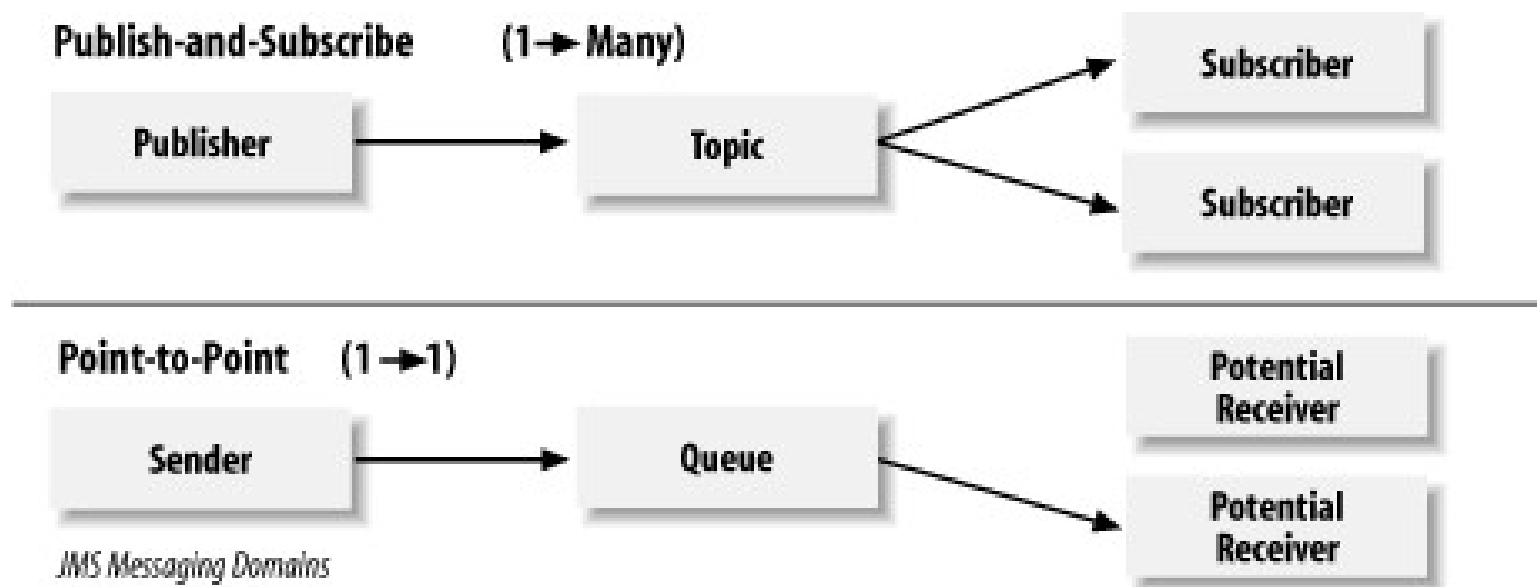
Arhitektura sistema sa redovima poruka (Message queuing systems)

- * Ako samo nekoliko rutera treba da zna za topologiju mreže, tada upravljač izvornog reda treba samo da zna kom susednom ruteru, recimo R, treba proslediti poruku, za dati određišni red.
- Kada pošiljalac A stavi poruku za odredište B u svoj lokalni red, ta poruka se prvo prenosi do najbližeg rutera, recimo R1. Ruter R1 može izvesti iz imena B da poruku treba proslediti ruteru R2. Na ovaj način, samo ruteri treba da budu ažurirani kada se redovi dodaju ili uklanjaju, dok svaki drugi menadžer redova mora znati samo gde je najbliži ruter.

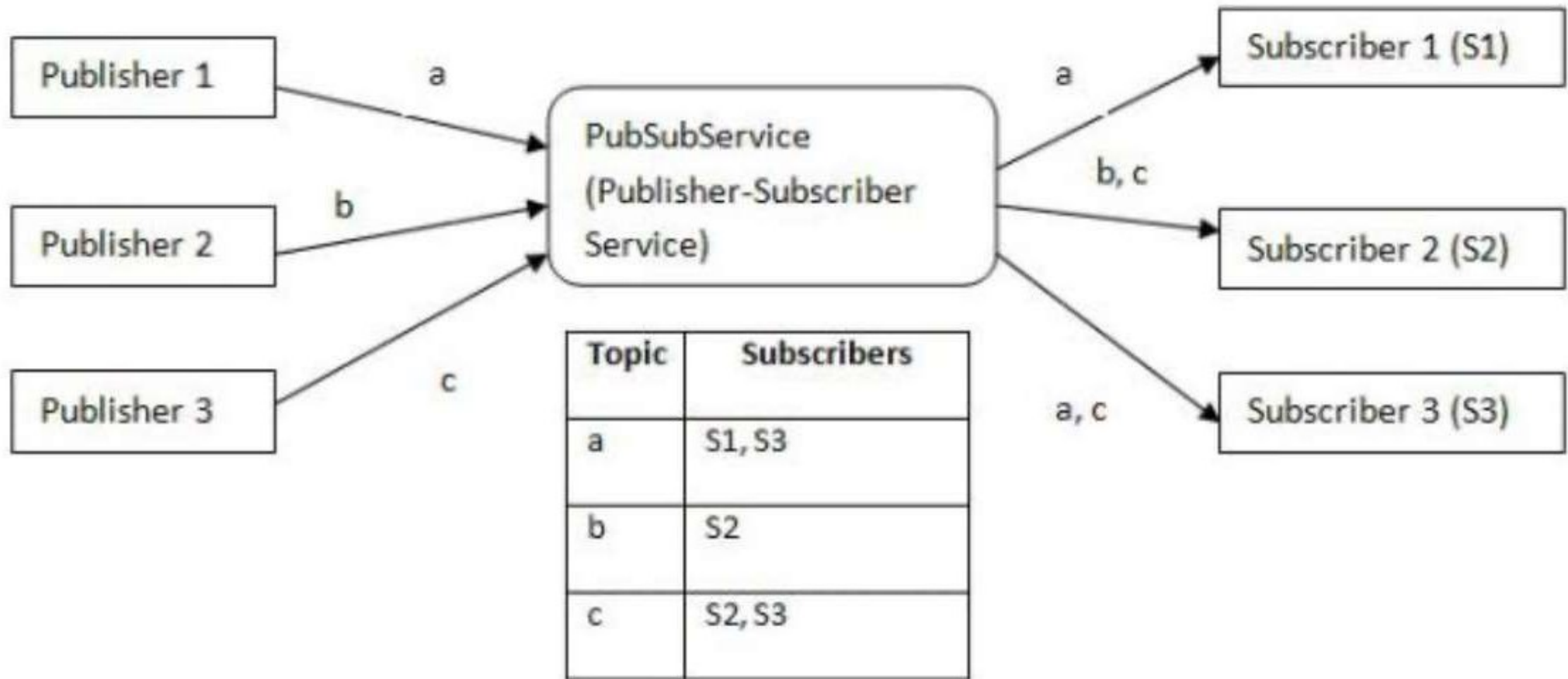


Publish-subscribe sistemi za razmenu poruka

- *Umesto pošiljaoca i primaoca imamo Publisher(Izdavač) i Subscriber(Pretplatnik)
- *Za razliku od Point-to-Point sistema gde se poruka prima samo od strane jednog primaoca kod Publish-Subscribe sistema poruka može biti primljena od više Subscriber-a
- * Publisher kreira poruku i publikuje (objavljuje) poruku u Topic-u (temi) (ovde je destinacija Topic umesto reda (queue))
- Više različitih Subscriber-a se može prijaviti (pretplatiti) za Topic i koristiti poruke koje su objavljene u tom Topic-u.
- * Subscriber može primiti poruku objavljenu u Topic-u, samo ako se Subscriber već prijavio za taj Topic. Svaka poruka poslata pre njegove prijave neće biti primljena od strane Subscriber-a
- *Implementacija ovog modela može naložiti i da poruka ne bude primljena ako Subscriber nije aktivan, mada većina implemetacija ne zahteva ovo ograničenje



Publish-subscribe sistemi za razmenu poruka



Publish-subscribe sistemi za razmenu poruka

- Višestruki pretplatnici (potrošači) mogu se pretplatiti na temu sa ili bez filtera
- Pretplate se prikupljaju pomoću komponente za raspoređivanja događaja (event dispatcher), odgovorne za usmeravanje događaja svim odgovarajućim pretplatnicima.
- Događaj sadrži informacije o određenom dešavanju ili promeni stanja unutar sistema, i izdavači distribuiraju događaje pretplatnicima koji su se pretplatili da primaju ažuriranja tih informacija.

Publish-subscribe sistemi za razmenu poruka

- U pretplati po temama, pretplatnici izražavaju interes za primanje događaja koji se odnose na određene teme.
 - Izdavači kategorizuju događaje u različite teme, a pretplatnici se pretplaćuju na teme koje ih zanimaju.
 - Pretplatnici primaju obaveštenja za sve događaje objavljene pod temama na koje su pretplaćeni.
- U pretplati po sadržaju, pretplatnici specificiraju kriterijume zasnovane na sadržaju događaja koje žele da prime.
 - Izdavači objavljuju događaje sa povezanim sadržajem, a pretplatnici postavljaju filtere koji definišu specifične attribute ili uslove koji ih interesuju.
 - Pretplatnici primaju samo događaje koji odgovaraju njihovim specifikovanim filterima.

Interfejs koji je na raspolaganju aplikacijama za razmenu poruka kod P/S sistema

Publish(event)

- * Izdavači koriste operaciju Publish(event) kako bi distribuirali događaje u sistem. Kada izdavač pozove ovu operaciju, emituje događaj svim pretplatnicima koji su izrazili interesovanje za događaje tog tipa.
 - Događaj može predstavljati bilo koje relevantne informacije ili ažuriranja koje izdavači žele da podele sa pretplatnicima.
 - Izdavači ne moraju znati identitete pojedinačnih pretplatnika; jednostavno objavljuju događaj u sistem.

Subscribe(filter):

- * Pretplatnici koriste operaciju subscribe(filter) kako bi izrazili interesovanje za primanje određenih tipova događaja. Kada pretplatnik pozove ovu operaciju, specificira filter koji definiše kriterijume za događaje koje želi da prima.
 - Filter može biti zasnovan na tipovima događaja, atributima ili bilo kojim drugim relevantnim kriterijumima. Pretplatnici primaju događaje koji odgovaraju navedenom filtru.

Interfejs koji je na raspolaganju aplikacijama za razmenu poruka kod P/S sistema

Notify(event):

- * Kada izdavač objavi događaj korišćenjem operacije publish(event) sistem za objavljivanje i pretplatu se pobrine da svi pretplatnici koji su se pretplatili sa filterima koji odgovaraju događaju dobiju obaveštenja.
 - Sistem isporučuje događaj ovim pretplatnicima, omogućavajući im da obrade ili reaguju na događaj prema potrebi.
 - Ovaj mehanizam obaveštavanja omogućava odvojenu komunikaciju između izdavača i pretplatnika.

Unsubscribe(filter):

- * Pretplatnici koriste operaciju unsubscribe(filter) kako bi povukli svoje interesovanje za primanje događaja koji odgovaraju određenom filteru.
 - Kada pretplatnik pozove ovu operaciju, sistem uklanja odgovarajuću pretplatu, osiguravajući da budući događaji koji odgovaraju filtru neće biti isporučeni tom pretplatniku.

Publish-subscribe sistemi za razmenu poruka

- * Ove operacije zajedno čine osnovne mehanizme komunikacije u sistemima za objavljivanje i pretplatu, olakšavajući fleksibilnu i odvojenu interakciju između izdavača i pretplatnika.
- * Izdavači mogu objaviti događaje bez potrebe da imaju informacije o pojedinačnim pretplatnicima, a pretplatnici mogu izraziti svoje interesovanje za relevantne događaje putem filtera.
- * U okviru modela objavljivanja/pretplatne, nema ograničenja u vezi sa ulogom klijenta; klijent može biti i proizvođač i potrošač teme
- * Ova odvojenost poboljšava skalabilnost, fleksibilnost i modularnost u distribuiranim sistemima.

Publish-subscribe sistemi

* Filtriranje po temi:

- Scenario: Razmotrimo sistem gde se objavljuju vesti, a korisnici mogu da se pretplate kako bi dobijali ažuriranja o određenim temama od interesa.
- Implementacija: Izdavači klasifikuju vesti u različite teme kao što su politika, sport, tehnologija, itd. Pretplatnici zatim biraju teme za koje su zainteresovani.

* Primer: Izdavač objavljuje članak o tehnološkim napretcima sa temom "Tehnologija".

- Pretplatnici koji su se pretplatili na temu "Tehnologija" dobijaju obaveštenja o ovom članku.

Publish-subscribe sistemi

* Filtriranje na osnovu sadržaja:

- Scenario: U sistemu za praćenje berze, trgovci su zainteresovani za primanje ažuriranja o akcijama koje ispunjavaju određene kriterijume kao što su promene cena ili fluktuacije volumena.
- Implementacija: Izdavači objavljuju ažuriranja za sve akcije na tržištu, ali pretplatnici žele da primaju obaveštenja samo o akcijama koje odgovaraju njihovim kriterijumima.

* Primer: Izdavač objavljuje ažuriranja za sve akcije uključujući njihovu trenutnu cenu i volumen.

- Pretplatnici specificiraju filtere kao što su "Obavesti me ako cena akcije Apple pređe 150 dolara" ili "Obavesti me ako se volumen akcije Tesla udvostruči".

Publish-subscribe sistemi

- * Sistem filtrira relevantna ažuriranja na osnovu kriterijuma pretplatnika i dostavlja obaveštenja u skladu sa tim.
- * U filtriranju po temi, pretplatnici izražavaju interesovanje za događaje koji se odnose na određene teme, dok u filtriranju po sadržaju, pretplatnici specificiraju kriterijume na osnovu sadržaja događaja koje žele da prime.

Java Messaging Service (JMS)

- * Java Messaging Service (JMS) je specifikacija standardizovanog načina za komunikaciju između distribuiranih Java programa.
- * Specifikacija objedinjuje paradigme objavljivanja i pretplate (publish-subscribe) i redova poruka (message queue) podržavajući teme (topics) i redove (queues) kao alternativne destinacije za poruke.

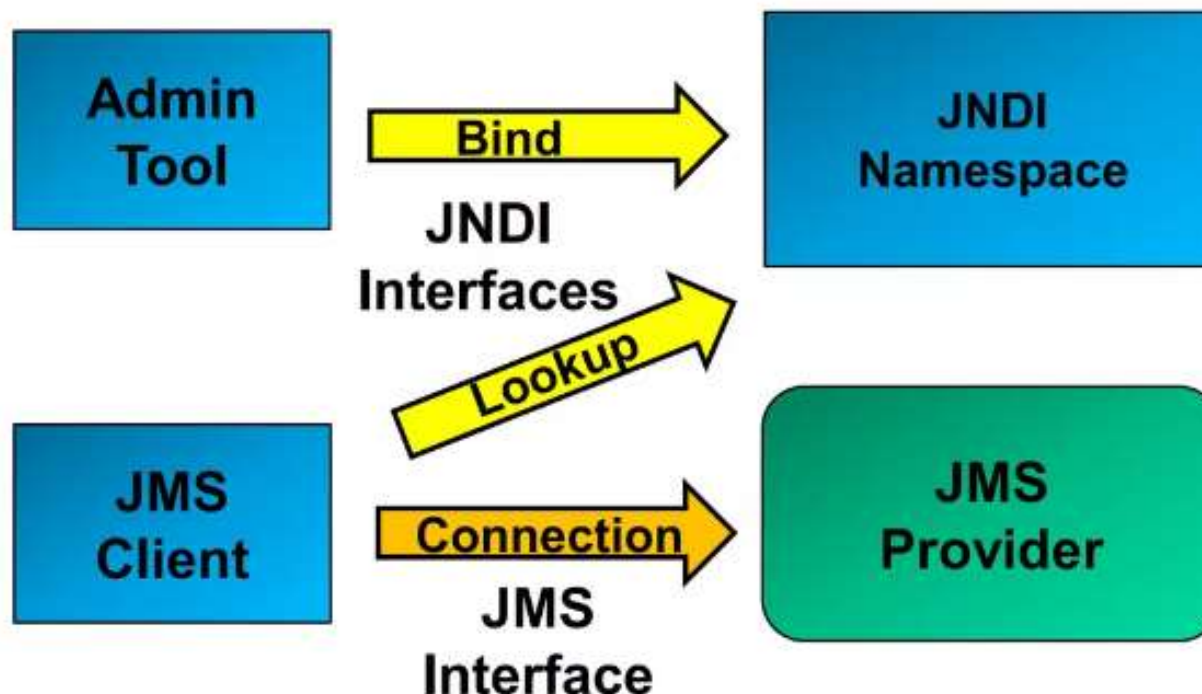
Java Messaging Service (JMS)

* JMS razlikuje sledeće ključne uloge:

- JMS klijent je Java program ili komponenta koja proizvodi ili konzumira poruke, JMS proizvođač je program koji kreira i proizvodi poruke, a JMS potrošač je program koji prima i konzumira poruke.
- JMS provajder je jedan od sistema koji implementiraju JMS specifikaciju.
- JMS poruka je objekat koji se koristi za komunikaciju informacija između JMS klijenata (od proizvođača ka potrošačima).
- JMS destinacija je objekat koji podržava komunikaciju u JMS-u. To je ili JMS tema ili JMS red.

Java Messaging Service (JMS)

- * JNDI (Java Name Directory Interface) provajder:
Sadrži povezivanja za fabrike konekcija i destinacije
(vraća odgovore na lookup zahteve na osnovu imena)
- * JMS provajder: Implementira JMS interfejs

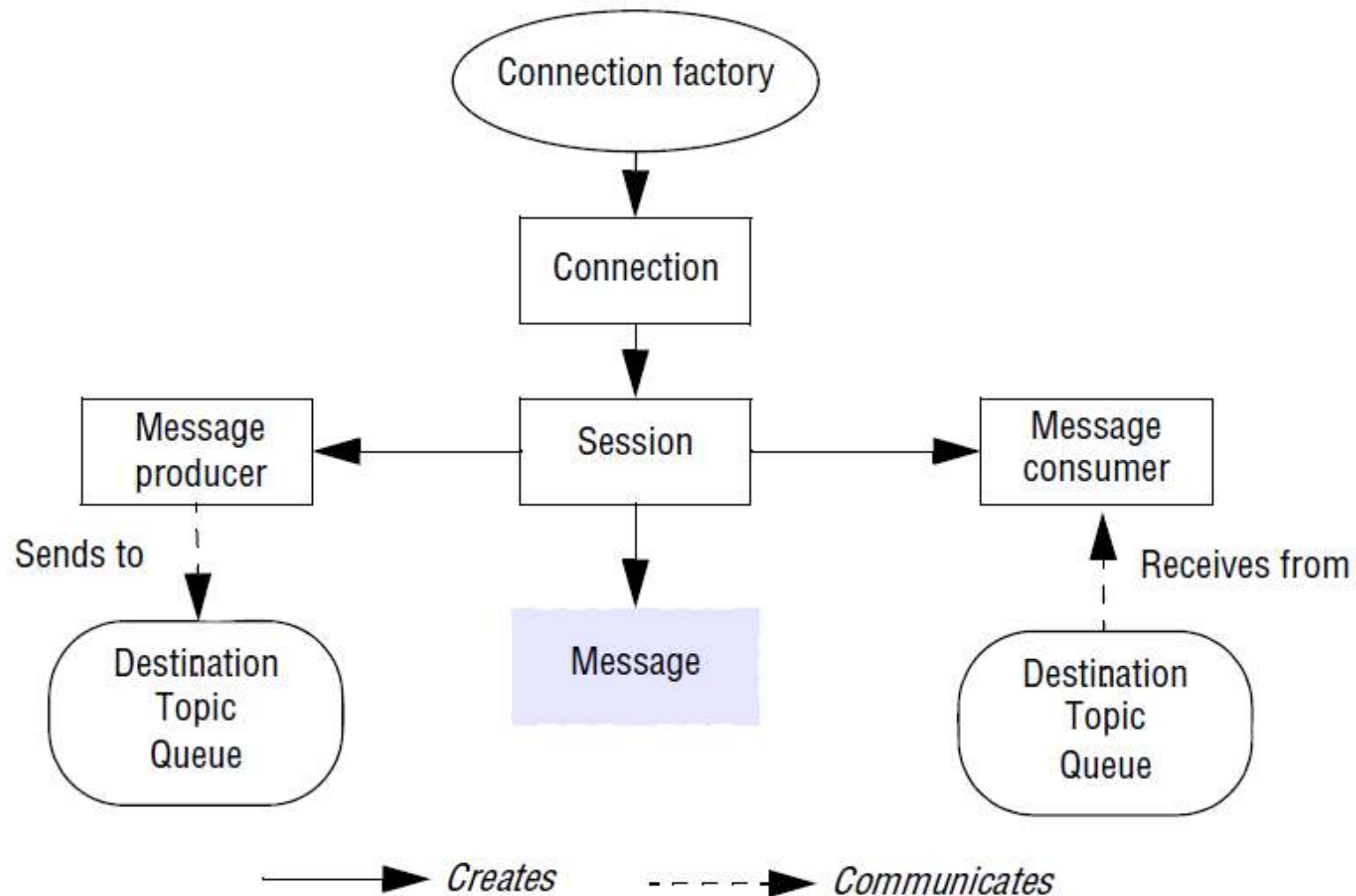


Java Messaging Service (JMS)

- * Administrativni objekti u JMS su unapred konfigurisani JMS objekti koje kreira administrator za upotrebu od strane klijenata.
- * Oni su unapred konfigurisani i upravljani od strane administratora, umesto da se dinamički kreiraju od strane kôda aplikacije.
- * Ovi objekti služe kao “most” između kôda klijenta i JMS provajdera.
 - Fabrike konekcija omogućavaju klijentima da uspostave konekcije sa JMS provajderom.
 - One enkapsuliraju detalje o kreiranju konekcija, sesija i proizvođača/potrošača poruka.
 - Korišćenjem fabrike konekcija, klijenti mogu da dobiju konekciju sa JMS provajderom bez potrebe da znaju detalje implementacije niskog nivoa.

Programiranje sa JMS-om

- fabrika konekcija i destinacije se koriste u JMS operacijama



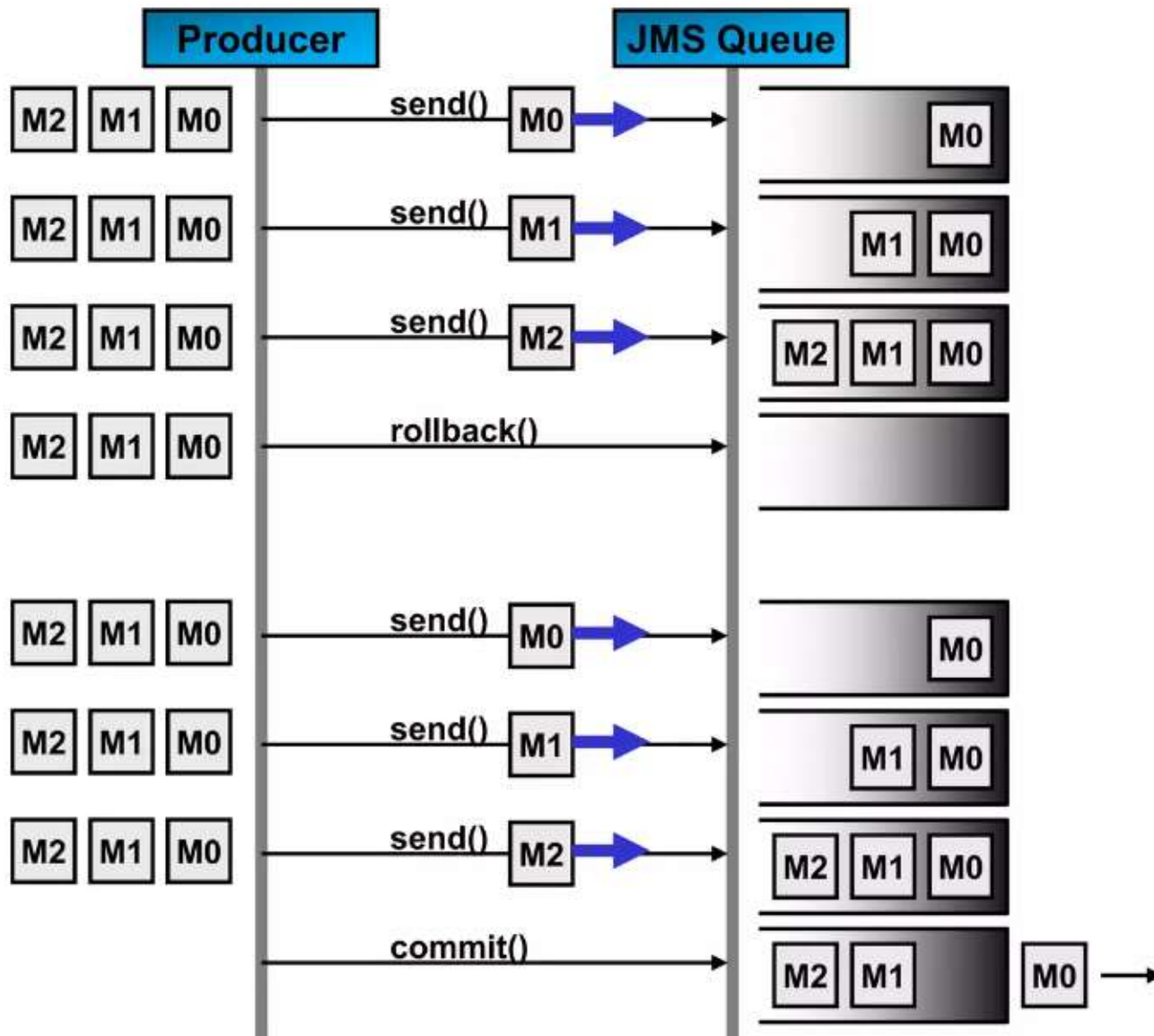
Programiranje sa JMS-om

- * Da bi se komuniciralo sa JMS provajderom, prvo je neophodno uspostaviti vezu između klijentskog programa i JMS provajdera.
- * Ovo se postiže kroz fabriku konekcija (servis odgovoran za kreiranje konekcija sa potrebnim osobinama).
 - Rezultujuća konekcija je logički kanal između klijenta i JMS provajdera.
- * Napomena: mogu se uspostaviti dva tipa veza, TopicConnection ili QueueConnection.

Programiranje sa JMS-om

- * Konekcije (veze) se mogu koristiti za kreiranje jedne ili više sesija – sesija je serija operacija koje uključuju kreiranje, proizvodnju i konzumaciju poruka vezanih za logički zadatak.
 - Rezultujući objekat sesije takođe podržava operacije za kreiranje transakcija (grupiše seriju operacija zajedno u jednu atomičnu), podržavajući sve-ili-ništa izvršavanje serije operacija.
- * Postoji jasna razlika između sesija za teme i sesija za redove, tako da TopicConnection može podržati jednu ili više sesija za teme, a QueueConnection može podržati jednu ili više sesija za redove.

Programiranje sa JMS-om (transakcije)

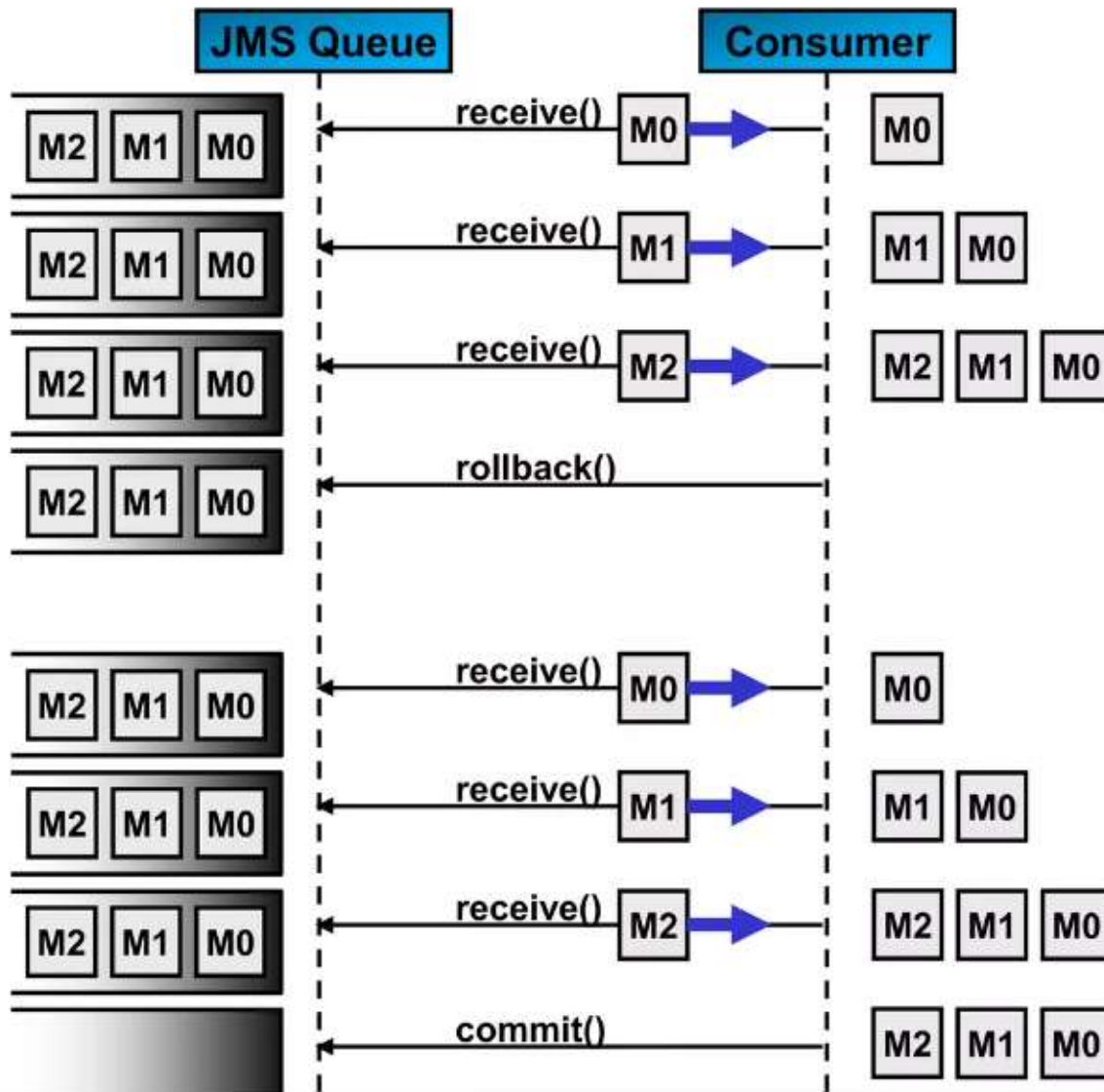


Ako se bilo koja operacija u transakciji ne izvrši, ona se poništava (roll back) tj. efekti svih operacija u transakciji se poništavaju

Rollback poništava sve poruke u JMS redu koje nisu poslate primaocu

Commit uzrokuje da JMS red pošalje sve neposlate poruke u transakciji

Programiranje sa JMS-om (transakcije)



Rollback šalje JMS redu instrukciju da ignoriše sve isporučene poruke (isporuka poruka kreće ponovo od M0)

Commit obaveštava JMS red da su sve poruke uspešno primljene od strane primaoca i poruke se brišu iz reda

Programiranje sa JMS-om

- * Objekat sesije je centralan za rad JMS-a, podržava metode za kreiranje poruka, kreiranje proizvođača poruka i kreiranje potrošača poruka.
- * U JMS-u, poruka se sastoji od tri dela: zaglavlja, seta svojstava(property) i tela poruke.
- * Zaglavlje sadrži sve informacije potrebne za identifikaciju i rutiranje poruke, uključujući odredište (referencu na temu ili red), prioritet poruke, datum isteka, ID poruke i vremensku oznaku.
 - Većina ovih polja se kreira od strane JMS provajdera, ali neka mogu biti specifično popunjena od strane korisnika.

Programiranje sa JMS-om

- * Svojstva (property) su definisana od strane korisnika i mogu se koristiti za povezivanje drugih metapodataka aplikacije sa porukom.
 - Na primer, ako se implementira sistem svestan konteksta, svojstva se mogu koristiti da se izraze dodatni kontekst koji je povezan sa porukom, uključujući polje za lokaciju.
- * U JMS-u, telo može biti: tekstualne poruka, niz bajtova, serijalizovani Java objekat, niz primitivnih Java vrednosti ili složeniji skup parova imena/vrednosti.
 - Message, TextMessage, StreamMessage, MapMessage, ObjectMessage i BytesMessage

Programiranje sa JMS-om

- * Proizvođač poruka je objekat koji se koristi za objavljivanje poruka pod određenom temom ili slanje poruka u red.
- * Potrošač poruka je objekat koji se koristi za pretplatu na poruke koje se tiču određene teme ili za primanje poruka iz reda.

Programiranje sa JMS-om

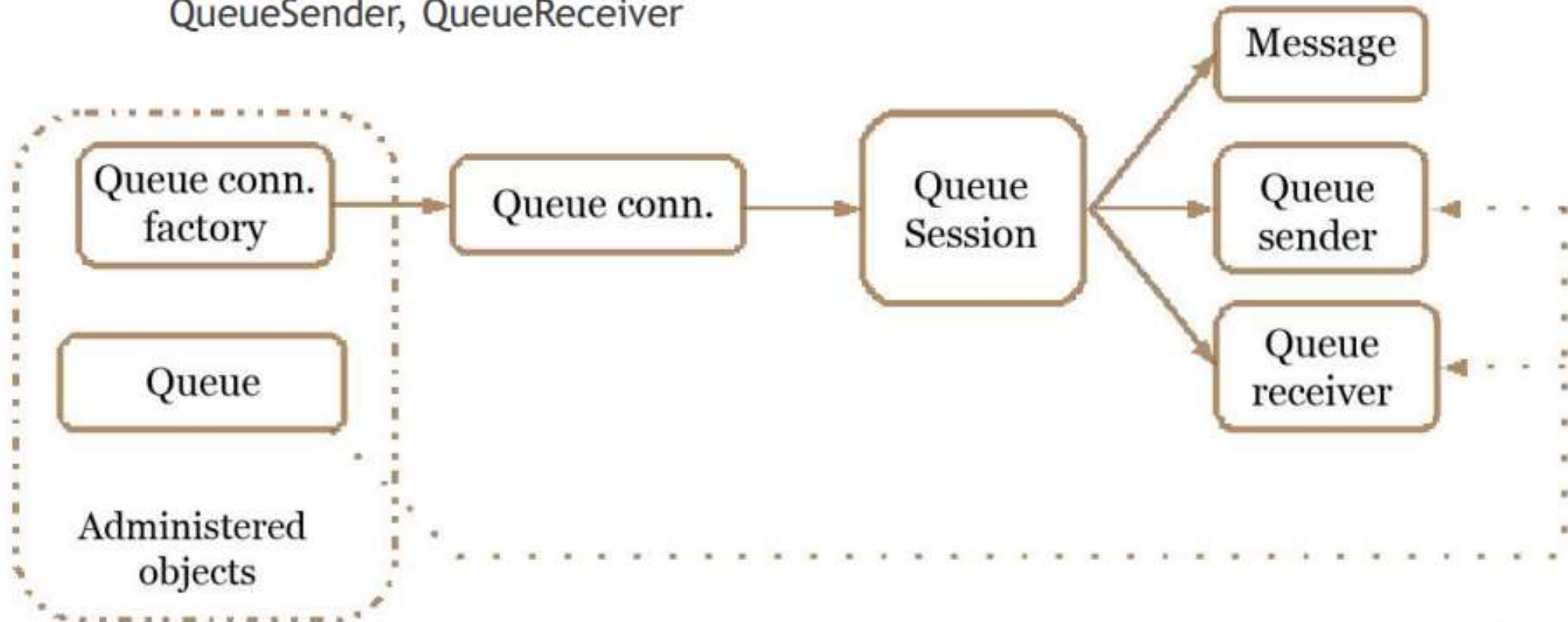
* **Potrošač je složeniji od proizvođača iz dva razloga.**

- Moguće je povezati filtere sa potrošačima poruka specificirajući ono što se naziva selektorom poruke - predikatom definisanim preko vrednosti u delovima zaglavlja i svojstava poruke (ne tela).
 - To se može koristiti, na primer, za filtriranje poruka sa određene lokacije u navedenom primeru svesti o kontekstu.
- Postoje dva moda za primanje poruka:
 - program može ili blokirati korišćenjem operacije primanja ili
 - može uspostaviti objekat slušaoca poruka (MessageListener) koji mora obezbediti metodu onMessage koja se poziva kada god se identifikuje odgovarajuća dolazeća poruka.

JMS P2P Implementacija

▶ Glavni interfejsi:

- ▶ QueueConnectionFactory, Queue, QueueConnection, QueueSession, Message, QueueSender, QueueReceiver



JMS P2P Implementacija

► Producer

- Obtain reference to QueueConnectionFactory,
- Get reference to Queue,
- Create QueueConnection,
- Create QueueSession,
- Create QueueSender,
- Create Message,
- Send Message.

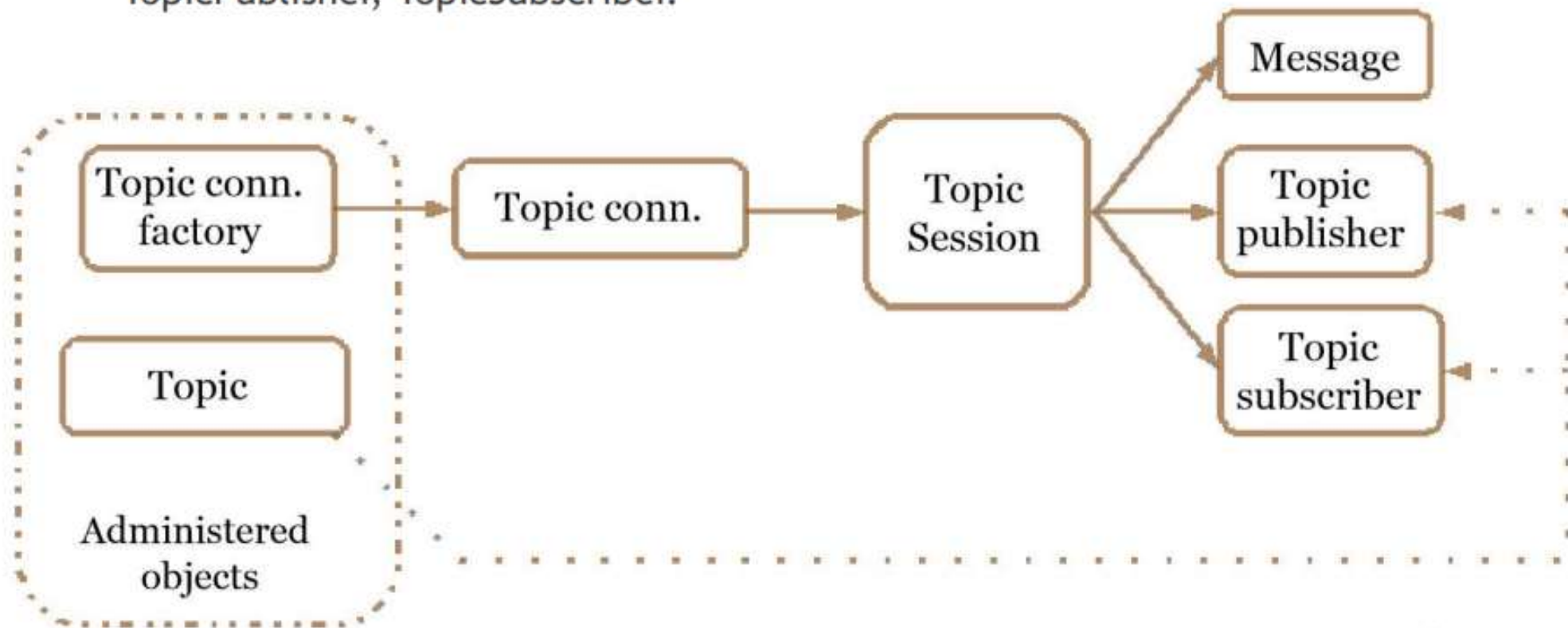
► Consumer

- Obtain reference to QueueConnectionFactory,
- Get reference to Queue,
- Create QueueConnection,
- Create QueueSession,
- Create QueueReceiver,
- Wait for message, implement interface MessageListener.

JMS Publish-and-Subscribe Implementacija

► Glavni interfejsi:

- TopicConnectionFactory, Topic, TopicConnection, TopicSession, Message, TopicPublisher, TopicSubscriber.



JMS Publish-and-Subscribe Implementacija

▶ Producer

- ▶ Obtain reference to *TopicConnectionFactory*,
- ▶ Get reference to *Topic*,
- ▶ Create *TopicConnection*,
- ▶ Create *TopicSession*,
- ▶ Create *TopicPublisher*,
- ▶ Create *Message*,
- ▶ Send *Message*.

▶ Consumer

- ▶ Obtain reference to *TopicConnectionFactory*,
- ▶ Get reference to *Topic*,
- ▶ Create *TopicConnection*,
- ▶ Create *TopicSession*,
- ▶ Create *TopicSubscriber*,
- ▶ Wait for message, implement interface *MessageListener*.

Različite implementacije

- Sistemi sa radovima poruka (i/ili Publish Subscribe sistemi) su implementirani u
 - ✓ JMS,
 - ✓ MSMQ
 - ✓ Apache ActiveMQ
 - ✓ Apache Kafka
 - ✓ Apache Pulsar
 - ✓ IBM MQ
 - ✓ NATS
 - ✓ RabbitMQ